Scheduling parallel dedicated machines with two competing agents

Ren-Xia Chen[a], Shi-Sheng Li[a,*]

[a]School of Mathematics and Information Science, Zhongyuan University of Technology, Zhengzhou 450007, People's Republic of China

Abstract: with

Keywords: Scheduling; Parallel dedicated machines; Two-agent; Pareto-optimal frontier; FPTAS

1 Introduction

2 Problem description and notations

The input consists of a set $\mathcal{M} = \{M_1, M_2, \ldots, M_m\}$ of m parallel dedicated machines (PDm), and two competing agents, referred to as agents A and B, respectively. Agent A is tasked with scheduling a set $\mathcal{J}^A$ of n_A independent jobs, while agent B has to schedule a set $\mathcal{J}^B$ of n_B independent jobs, where $\mathcal{J}^A \cap \mathcal{J}^B = \emptyset$. All jobs are available for processing at time zero, and no preemption is permitted. For $X \in \{A, B\}$, the set $\mathcal{J}^X$ is divided into m disjoint subsets $\mathcal{J}_1^X, \mathcal{J}_2^X, \ldots, \mathcal{J}_m^X$ in advance. Each subset $\mathcal{J}_i^X$ contains n_i^X jobs, which must be processed exclusively on machine M_i, $i = 1, 2, \ldots, m$, where $n_X = \sum_{i=1}^m n_i^X$. Each job, denoted as $J_{i,j}^X$, is characterized by an agent index X, $X = A, B$, a machine index i, $i = 1, 2, \ldots, m$, and a job index j, $j = 1, 2, \ldots, n_i^X$. Let $p_{i,j}^X > 0$ be an integer representing the *processing time* of job $J_{i,j}^X$. In addition, when applicable, let $d_{i,j}^X \geq 0$ denote the *due date* and $w_{i,j}^X > 0$ represent the *weight* of job $J_{i,j}^X$.

A job schedule for the jobs in $\mathcal{J}^A \cup \mathcal{J}^B$ is defined by a job processing sequence on each machine M_i ($i = 1, 2, \ldots, m$) for the jobs in $\mathcal{J}_i^A \cup \mathcal{J}_i^B$ and their corresponding starting times. Moreover, for $X \in \{A, B\}$ and $i = 1, 2, \ldots, m$, the following notations are introduced:

- $\mathcal{J} = \mathcal{J}^A \cup \mathcal{J}^B$: the set of all jobs in $\mathcal{J}^A \cup \mathcal{J}^B$.

*Corresponding author. E-mail address: liss@zut.edu.cn (S.S. Li)

- $n = n_A + n_B$: the total number of jobs in $\mathcal{J}$.
- $\mathcal{J}_i = \mathcal{J}_i^A \cup \mathcal{J}_i^B$: the set of all jobs in $\mathcal{J}_i^A \cup \mathcal{J}_i^B$.
- $n_i = n_i^A + n_i^B$: the total number of jobs in $\mathcal{J}_i$.
- $P_i^X = \sum_{j=1}^{n_i^X} p_{i,j}^X$: the total processing time of jobs in $\mathcal{J}_i^X$.
- $P_i = P_i^A + P_i^B$: the total processing time of jobs in $\mathcal{J}_i$.
- $P_{\max}^X = \max\limits_{1 \le i \le m} \{P_i^X\}$: the maximum value among $P_1^X, P_2^X, \ldots, P_m^X$.
- $P_{\max} = \max\limits_{1 \le i \le m} \{P_i\}$: the maximum value among $P_1, P_2, \ldots, P_m$.
- $P(X) = \sum_{J_{i,j}^X \in \mathcal{J}^X} p_{i,j}^X$: the total processing time of jobs in $\mathcal{J}^X$.
- $W(X) = \sum_{J_{i,j}^X \in \mathcal{J}^X} w_{i,j}^X$: the total weight of jobs in $\mathcal{J}^X$.
- $\mathcal{N}_i = \mathcal{J}_1 \cup \mathcal{J}_2 \cup \cdots \cup \mathcal{J}_i$: the set of jobs that must be processed on the first i machines.
- $N_i = |\mathcal{N}_i| = \sum_{k=1}^{i} n_i$: the total number of jobs in $\mathcal{N}_i$.
- $\mathcal{J}_{i,j}^X = \{J_{i,1}^X, J_{i,2}^X, \ldots, J_{i,j}^X\}$: the set of the first j jobs in $\mathcal{J}_i^X$ for $j = 1, 2, \ldots, n_i^X$.
- $P_{i,j}^X = \sum_{k=1}^{j} p_{i,k}^X$: the total processing times of jobs in $\mathcal{J}_{i,j}^X$.

To simplify the presentation, define $\mathcal{N}_0 = \emptyset$, $N_0 = 0$, $n_0 = 0$, $n_0^X = 0$, $\mathcal{J}_{i,0}^X = \emptyset$, and $P_{i,0}^X = 0$. Furthermore, for each job $J_{i,j}^X$, the following notations and terminologies are employed in a given schedule π:

- $C_{i,j}^X(\pi)$: the completion time of $J_{i,j}^X$ in π.
- $U_{i,j}^X(\pi)$: the tardy indicator variable of $J_{i,j}^X$ in π, where $U_{i,j}^X(\pi) = 1$ if $C_{i,j}^X(\pi) > d_{i,j}^X$, and $U_{i,j}^X(\pi) = 0$ otherwise.
- $Y_{i,j}^X(\pi) = \min\{p_{i,j}^X, \max\{C_{i,j}^X(\pi) - d_{i,j}^X, 0\}\}$: the late work of $J_{i,j}^X$ in π.
- $J_{i,j}^X$ is referred to as an *early* job in π if $C_{i,j}^X(\pi) \le d_{i,j}^X$. In this case, $U_{i,j}^X(\pi) = 0$ and $Y_{i,j}^X(\pi) = 0$.
- $J_{i,j}^X$ is referred to as a *partially early* job in π if $d_{i,j}^X < C_{i,j}^X(\pi) < d_{i,j}^X + p_{i,j}^X$. In this case, $U_{i,j}^X(\pi) = 0$ and $0 < Y_{i,j}^X(\pi) < p_{i,j}^X$.
- $J_{i,j}^X$ is referred to as a *tardy* job in π if $C_{i,j}^X(\pi) > d_{i,j}^X$. In this case, $U_{i,j}^X(\pi) = 1$ and $Y_{i,j}^X(\pi) > 0$.
- $J_{i,j}^X$ is referred to as a *late* job in π if $C_{i,j}^X(\pi) \ge d_{i,j}^X + p_{i,j}^X$. In this case, $Y_{i,j}^X(\pi) = p_{i,j}^X$.

- $J_{i,j}^X$ is referred to as a *non-late* job in π if $C_{i,j}^X(\pi) < d_{i,j}^X + p_{i,j}^X$. Note that a job is non-late when it is either early or partially early.

Given a feasible schedule, its quality is evaluated by two scheduling criteria, each associated with a different agent. Agent X ($X \in A, B$) seeks to minimize its cost function f^X, which is based solely on the completion times of its own jobs. The criteria for each agent studied in this paper is chosen from one of the three possibilities:

- $\mathrm{TC}^X(\pi) = \sum_{i=1}^{m} \sum_{j=1}^{n_i^X} C_{i,j}^X(\pi)$: the total completion time of agent X.
- $\mathrm{TWU}^X(\pi) = \sum_{i=1}^{m} \sum_{j=1}^{n_i^X} w_{i,j}^X U_{i,j}^X(\pi)$: the weighted number of tardy jobs of agent X. In particular, when all jobs of agent X have unit weight, $\mathrm{TU}^X(\pi) = \sum_{i=1}^{m} \sum_{j=1}^{n_i^X} U_{i,j}^X(\pi)$ is used to denote the number of tardy jobs of agent X.
- $\mathrm{TY}^X(\pi) = \sum_{i=1}^{m} \sum_{j=1}^{n_i^X} Y_{i,j}^X(\pi)$: the total late work of agent X.

Without introducing ambiguity, the notation π can be omitted. A feasible schedule π is considered *Pareto-optimal* (PO) with respect to f^A and f^B if no other schedule π' exists satisfying $f^A(\pi') \leq f^A(\pi)$ and $f^B(\pi') \leq f^B(\pi)$, where at least one of these inequalities is strict. In this setting, $(f^A(\pi), f^B(\pi))$ is called a *PO point*. A set $\mathcal{X}$ of points forms a *PO frontier* if it includes all PO points.

In this paper, we address the *Pareto* variant of the parallel dedicated machine scheduling problem involving two competing agents. This variant aims to determine the PO frontier, minimizing both cost functions f^A and f^B. Following the scheduling framework outlined in Agnetis et al. (2014), this variant is denoted as $PDm|CO|\#(f^A, f^B)$, where $f^X \in \{\mathrm{TC}^X, \mathrm{TWU}^X, \mathrm{TY}^X\}$ for $X = A, B$. Note that when $m = 1$, problem $PD1|CO|\#(f^A, f^B)$ degenerates to the single-machine problem $1|CO|\#(f^A, f^B)$.

In the algorithm design, we also focus on the approximate PO frontier. With respect to problem $PDm|CO|\#(f^A, f^B)$, a set $\mathcal{X}_{(\rho_A,\rho_B)}$ of points is called a (ρ_A, ρ_B)-*approximate PO frontier* if, (i) for every PO point (z_A, z_B) in $\mathcal{X}$, there exists a point $(z'_A, z'_B) \in \mathcal{X}_{(\rho_A,\rho_B)}$ satisfying $z'_A \leq \rho_A z_A$ and $z'_B \leq \rho_B z_B$, and (ii) every point (z'_A, z'_B) in $\mathcal{X}_{(\rho_A,\rho_B)}$ is achievable, i.e., there exists a feasible schedule π satisfying $f^A(\pi) = z'_A$ and $f^B(\pi) = z'_B$.

Moreover, a closely related counterpart is the *restricted* variant $PDm|CO, f^B \leq Q|f^A$, which seeks a feasible schedule minimizing f^A while ensuring that $f^B \leq Q$, where Q is an upper bound on f^B. As indicated in T'kindt and Billaut (2006), if the restricted variant $PDm|CO, f^B \leq Q|f^A$ is binary (or unary) $\mathcal{NP}$-hard, then the Pareto problem $PDm|CO|\#(f^A, f^B)$ is also binary (or unary) $\mathcal{NP}$-hard.

3 General properties

In this section, we establish several foundational properties of PO schedules across different problem variants of $PDm|CO|\#(f^A, f^B)$, which will guide the development of algorithms for specific problems.

A cost function f^X to be minimized is termed *regular* if it is non-decreasing with respect to agent X's completion times $C^X_{i,j}$, $i = 1, 2, \ldots, m$ and $j = 1, 2, \ldots, n^X_i$. Since all scheduling criteria examined in this paper are regular, we only focus on schedules where, on each machine M_i $(i = 1, 2, \ldots, m)$, the jobs in $\mathcal{J}_i$ start at time zero and are processed consecutively without any idle times.

Lemma 3.1. *For each PO point of problem $PDm|CO|\#(f^A, f^B)$, with one criterion being the total completion time, i.e., $f^X = \mathrm{TC}^X$ for some $X \in \{A, B\}$, and the other criterion f^Z being regular, there exists a corresponding PO schedule where, on each machine M_i $(i = 1, 2, \ldots, m)$, the jobs in $\mathcal{J}^X_i$ are processed in a non-decreasing order of $p^X_{i,j}$, i.e., following the shortest processing time (SPT) order.*

Proof. Let π denote a PO schedule. If, for each $i = 1, 2, \ldots, m$, the jobs in $\mathcal{J}^X_i$ are processed on M_i in the SPT order in π, then the desired property holds. Otherwise, there exists a machine index $r \in \{1, 2, \ldots, m\}$ and two distinct job indices $k, l \in \{1, 2, \ldots, n^X_r\}$ such that $p^X_{r,k} > p^X_{r,l}$, and $J^X_{r,k}$ is processed before $J^X_{r,l}$ on M_r without any other job belonging to $\mathcal{J}^X_r$ in between. We can create a new schedule π' from π by swapping the positions of jobs $J^X_{r,k}$ and $J^X_{r,l}$ on M_r, while keeping the positions of all other jobs unchanged. In this case, we have $C^X_{r,l}(\pi') = C^X_{r,k}(\pi) - p^X_{r,k} + p^X_{r,l}$, $C^X_{r,k}(\pi') = C^X_{r,l}(\pi)$, and the completion times of all other jobs do not increase. Consequently, we have $f^X(\pi') = \mathrm{TC}^X(\pi') = \mathrm{TC}^X(\pi) + p^X_{r,l} - p^X_{r,k} < TC^X(\pi) = f^X(\pi)$. Since f^Z is regular, this implies $f^Z(\pi') \leq f^Z(\pi)$. This contradicts the assumption that π is a PO schedule. □

Lemma 3.2. *For each PO point of problem $PDm|CO|\#(f^A, f^B)$, with one criterion being the weighted number of tardy jobs, i.e., $f^X = \mathrm{TWU}^X$ for some $X \in \{A, B\}$, and the other criterion f^Z being regular, there exists a corresponding PO schedule where, on each machine M_i $(i = 1, 2, \ldots, m)$, (i) the early jobs in $\mathcal{J}^X_i$ are processed in a non-decreasing order of $d^X_{i,j}$, i.e., following the earliest due date (EDD) order, and (ii) all tardy jobs in $\mathcal{J}^X_i$ are processed last in an arbitrary order.*

Proof. Since the cost of a tardy job is a constant and the criteria of both agents are regular, property (ii) holds clearly. We only need to prove the correctness of property (i). Let π be a PO schedule that satisfies property (ii). If, for each $i = 1, 2, \ldots, m$, the early jobs in $\mathcal{J}^X_i$ are processed on M_i in the EDD order in π, then property (i) holds as well. Otherwise, there exists a machine index $r \in \{1, 2, \ldots, m\}$ and two distinct job indices $k, l \in \{1, 2, \ldots, n^X_r\}$ such that $d^X_{r,k} > d^X_{r,l}$, both $J^X_{r,k}$ and $J^X_{r,l}$ are early jobs, and $J^X_{r,k}$ is

processed before $J_{r,l}^X$ on M_r. This implies that $C_{r,l}^X(\pi) \leq d_{r,l}^X$. We can create a new schedule π' from π by shifting $J_{r,k}^X$ to the position immediately after $J_{r,l}^X$ on M_r, while keeping the positions of all other jobs unchanged. In π', we have $C_{r,l}^X(\pi') = C_{r,l}^X(\pi) - p_{r,k}^X < C_{r,l}^X(\pi) \leq d_{r,l}^X$, $C_{r,k}^X(\pi') = C_{r,l}^X(\pi) \leq d_{r,l}^X < d_{r,k}^X$, and the completion times of all other jobs do not increase. Consequently, we have $f^X(\pi') = \mathrm{TWU}^X(\pi') = \mathrm{TWU}^X(\pi) = f^X(\pi)$. Since f^Z is regular, this implies $f^Z(\pi') \leq f^Z(\pi)$. This implies that π' is also a PO schedule. By continuing the above shifting procedure successively, we can get a PO schedule satisfying both properties (i) and (ii). □

Lemma 3.3. *For each PO point of problem $PDm|CO|\#(f^A, f^B)$, with one criterion being the total late work, i.e., $f^X = \mathrm{TY}^X$ for some $X \in \{A, B\}$, and the other criterion f^Z being regular, there exists a corresponding PO schedule where, on each machine M_i ($i = 1, 2, \ldots, m$), (i) the non-late jobs in $\mathcal{J}_i^X$ are processed in the EDD order, and (ii) all late jobs in $\mathcal{J}_i^X$ are processed last in an arbitrary order.*

Proof. Since the cost of a non-late job is a constant and the criteria of both agents are regular, property (ii) holds clearly. We will focus on proving property (i). Let π be a PO schedule that satisfies property (ii). If, for each $i = 1, 2, \ldots, m$, the non-late jobs in $\mathcal{J}_i^X$ are processed on M_i in the EDD order in π, then property (i) holds as well. Otherwise, there exists a machine index $r \in \{1, 2, \ldots, m\}$ and two distinct job indices $k, l \in \{1, 2, \ldots, n_r^X\}$ such that $d_{r,k}^X > d_{r,l}^X$, both $J_{r,k}^X$ and $J_{r,l}^X$ are non-late jobs, and $J_{r,k}^X$ is processed before $J_{r,l}^X$ on M_r without any other job belonging to $\mathcal{J}_r^X$ in between. This implies that $Y_{r,k}^X(\pi) = 0$ and $Y_{r,l}^X(\pi) = \min\{p_{r,l}^X, \max\{C_{r,l}^X(\pi) - d_{r,l}^X, 0\}\} = \max\{C_{r,l}^X(\pi) - d_{r,l}^X, 0\} < p_{r,l}^X$. We can create a new schedule π' from π by shifting $J_{r,k}^X$ to the position immediately after $J_{r,l}^X$ on M_r, while keeping the positions of all other jobs unchanged. In π', we have $C_{r,l}^X(\pi') = C_{r,l}^X(\pi) - p_{r,k}^X$, $C_{r,k}^X(\pi') = C_{r,l}^X(\pi)$, and the completion times of all other jobs do not increase. Since f^Z is regular, this implies $f^Z(\pi') \leq f^Z(\pi)$. Next, we show that $f^X(\pi') = \mathrm{TY}^X(\pi') \leq \mathrm{TY}^X(\pi) = f^X(\pi)$. This is equivalent to demonstrating that $Y_{r,k}^X(\pi') + Y_{r,l}^X(\pi') \leq Y_{r,k}^X(\pi) + Y_{r,l}^X(\pi)$. We consider two cases:

Case 1. $J_{r,l}^X$ is an early job in π'. In this case, $Y_{r,l}^X(\pi') = 0$. Since $J_{r,l}^X$ is non-late in π and $d_{r,k}^X > d_{r,l}^X$, $J_{r,k}^X$ is non-late in π'. Hence, we have $Y_{r,k}^X(\pi') + Y_{r,l}^X(\pi') = Y_{r,k}^X(\pi') = \max\{C_{r,k}^X(\pi') - d_{r,k}^X, 0\} \leq \max\{C_{r,l}^X(\pi) - d_{r,l}^X, 0\} = Y_{r,l}^X(\pi) = Y_{r,k}^X(\pi) + Y_{r,l}^X(\pi)$.

Case 2. $J_{r,l}^X$ is a partially early job in π'. In this case, $J_{r,l}^X$ is a partially early job in π. Then $Y_{r,l}^X(\pi') = C_{r,l}^X(\pi') - d_{r,l}^X = C_{r,l}^X(\pi) - p_{r,k}^X - d_{r,l}^X$ and $Y_{r,l}^X(\pi) = C_{r,l}^X(\pi) - d_{r,l}^X$. Since $d_{r,k}^X > d_{r,l}^X$, $J_{r,k}^X$ is a non-late job in π'. Hence, we have $Y_{r,k}^X(\pi') + Y_{r,l}^X(\pi') < p_{r,k}^X + Y_{r,l}^X(\pi') = p_{r,k}^X + C_{r,l}^X(\pi) - p_{r,k}^X - d_{r,l}^X \leq \max\{C_{r,l}^X(\pi) - d_{r,l}^X, 0\} = Y_{r,l}^X(\pi) = Y_{r,k}^X(\pi) + Y_{r,l}^X(\pi)$.

In both cases, we conclude that $Y_{r,k}^X(\pi') + Y_{r,l}^X(\pi') \leq Y_{r,k}^X(\pi) + Y_{r,l}^X(\pi)$. This further implies that π' is also a PO schedule. By continuing the above shifting procedure successively, we can get a PO schedule satisfying both properties (i) and (ii). □

When both agents want to minimize the weighted number of tardy jobs, the following lemma outlines the processing sequence of early jobs of both agents on each machine. Since it is an intuitive generalization of the result provided by Agnetis et al. (2004) for problem $1|CO, \mathrm{TU}^B \leq Q_B|\mathrm{TU}^A$, the proof is omitted.

Lemma 3.4. *For each PO point of problem $PDm|CO|\#(\mathrm{TWU}^A, \mathrm{TWU}^B)$, there exists a corresponding PO schedule where, on each machine M_i ($i = 1, 2, \ldots, m$), (i) the early jobs in $\mathcal{J}_i$ are processed in a non-decreasing order of their due dates $d_{i,j}^X$, i.e., following the EDD order, and (ii) all tardy jobs in $\mathcal{J}_i$ are processed last in an arbitrary order.*

For a given schedule π, if $f^X = \mathrm{TC}^X$, then all jobs of agent X are defined as *valid*; if $f^X = \mathrm{TWU}^X$, then all early jobs of agent X are defined as *valid*; if $f^X = \mathrm{TY}^X$, then all non-late jobs of agent X are defined as *valid*. Jobs not valid in π are defined as *invalid*.

4 Optimization algorithms

In this section, we address six problems, each defined by a pair of scheduling criteria f^A and f^B. Each f^X ($X = A, B$) is drawn from the set $\{\mathrm{TC}^X, \mathrm{TWU}^X, \mathrm{TY}^X\}$, encompassing all possible combinations of these criteria. Since problems $1||\mathrm{TWU}$ (Karp, 1972), $1||\mathrm{TY}$ (Potts and Van Wassenhove, 1992), and $1|CO, \mathrm{TC}^B \leq Q|\mathrm{TC}^A$ (Agnetis et al., 2004) are $\mathcal{NP}$-hard, all six of these problems are at least $\mathcal{NP}$-hard as well. We develop pseudo-polynomial DP algorithms for six problems, demonstrating that they are all $\mathcal{NP}$-hard in the ordinary sense.

In our algorithm design, the two scheduling criteria TWU^X and TY^X exhibit similar properties, allowing them to be addressed using a unified method. Let $\mathrm{TG} \in \{\mathrm{TWU}, \mathrm{TY}\}$. Then $\mathrm{TG}^X \in \{\mathrm{TWU}^X, \mathrm{TY}^X\}$.

Moreover, to analyze the time complexity of each designed algorithm, we define

$$\Delta_A = \begin{cases} \sum_{i=1}^{m}(n_i^A P_i^B + \sum_{j=1}^{n_i^A}(n_i^A - j + 1)p_{i,j}^A), & \text{if } f^A = \mathrm{TC}^A; \\ W(A), & \text{if } f^A = \mathrm{TWU}^A; \\ P(A), & \text{if } f^A = \mathrm{TY}^A. \end{cases} \tag{1}$$

and

$$\Delta_B = \begin{cases} \sum_{i=1}^{m}(n_i^B P_i^A + \sum_{j=1}^{n_i^B}(n_i^B - j + 1)p_{i,j}^B), & \text{if } f^B = \mathrm{TC}^B; \\ W(B), & \text{if } f^A = \mathrm{TWU}^B; \\ P(B), & \text{if } f^A = \mathrm{TY}^B. \end{cases} \tag{2}$$

Clearly, Δ_X serves as a valid upper bound on the objective function f^X of agent X.

4.1 Problem $PDm|CO|\#(\mathrm{TC}^A, \mathrm{TC}^B)$

Given the results in Lemma 3.1, we focus on schedules for problem $PDm|CO|\#(\mathrm{TC}^A, \mathrm{TC}^B)$ that obey the following constraints for each machine M_i $(i = 1, 2, \ldots, m)$: (i) the jobs in $\mathcal{J}_i^A$ are processed according to their SPT order; and (ii) the jobs in $\mathcal{J}_i^B$ are also processed according to their SPT order.

Our DP algorithm consists of m stages, each corresponding to a distinct machine, with $n_i^A n_i^B + 1$ iterations conducted during the i-th stage. Specifically, in each stage-iteration denoted as (i, j_A, j_B), where $i = 1, 2, \ldots, m$, $j_A = 0, 1, \ldots, n_i^A$, and $j_B = 0, 1, \ldots, n_i^B$, a state space $\mathcal{L}(i, j_A, j_B)$ is generated. Each state in $\mathcal{L}(i, j_A, j_B)$ is a 2-tuple (q_A, q_B), representing a feasible schedule for jobs in $\mathcal{N}_{i-1} \cup \mathcal{J}_{i,j_A}^A \cup \mathcal{J}_{i,j_B}^B$. Here, q_A denotes agent A's total completion time of the partial schedule, and q_B indicates agent B's total completion time of the partial schedule. In the dynamic update step, $\mathcal{L}(i, j_A, j_B)$ is generated from the previous sets $\mathcal{L}(i, j_A - 1, j_B)$ and $\mathcal{L}(i, j_A, j_B - 1)$.

The DP algorithm is initialized by setting $\mathcal{L}(0, 0, 0) = \{(0, 0)\}$. Let (q_A, q_B) be a given state in $\mathcal{L}(i, j_A, j_B)$. If $j_A \leq n_i^A - 1$ and $j_B \leq n_i^B - 1$, then two cases should be considered to generate a new state:

- **Case 1.** J_{i,j_A+1}^A is the next scheduled job on M_i. In this context, the contribution of J_{i,j_A+1}^A to agent A's objective value is $P_{i,j_A+1}^A + P_{i,j_B}^B$, representing the completion time of J_{i,j_A+1}^A on M_i. Hence, $(q_A + P_{i,j_A+1}^A + P_{i,j_B}^B, q_B)$ is added to $\mathcal{L}(i, j_A + 1, j_B)$.
- **Case 2.** J_{i,j_B+1}^B is the next scheduled job on M_i. In this context, the contribution of J_{i,j_B+1}^B to agent B's objective value is $P_{i,j_A}^A + P_{i,j_B+1}^B$, representing the completion time of J_{i,j_B+1}^B on M_i. Hence, $(q_A, q_B + P_{i,j_A}^A + P_{i,j_B+1}^B)$ is added to $\mathcal{L}(i, j_A, j_B + 1)$.

Note that once $\mathcal{L}(i, n_i^A, n_i^B)$ has been constructed and $i \leq m-1$, the i-th stage terminates, after which the algorithm proceeds to the $(i+1)$-th stage. This initialization involves setting $\mathcal{L}(i + 1, 0, 0) = \mathcal{L}(i, n_i^A, n_i^B)$. After completing all m stages, we can extract all PO points from $\mathcal{L}(m, n_m^A, n_m^B)$.

Moreover, we can apply the following elimination rule, which is straightforward to prove and will be employed to reduce the state space $\mathcal{L}(i, j_A, j_B)$.

Lemma 4.1. *For any two states (q_A, q_B) and (q'_A, q'_B) in $\mathcal{L}(i, j_A, j_B)$ satisfying $q_A \leq q'_A$ and $q_B \leq q'_B$, the second one can be deleted.*

During the construction of $\mathcal{L}(i, j_A, j_B)$, multiple states may have the same q_A (or q_B) value. Based on Lemma 4.1, one of the following elimination rules can be applied, depending on whether Δ_A or Δ_B is larger.

Rule R1. Among all elements in $\mathcal{L}(i, j_A, j_B)$ sharing the same q_A value, retain only the one with minimal q_B value (this rule is selected if $\Delta_A \leq \Delta_B$).

Rule R2. Among all elements in $\mathcal{L}(i, j_A, j_B)$ sharing the same q_B value, retain only the one with minimal q_A value (this rule is selected if $\Delta_A > \Delta_B$).

Based on the preceding analysis, we formulate the optimization algorithm (called **DP-TC-TC**) for problem $PDm|CO|\#(\mathrm{TC}^A, \mathrm{TC}^B)$ as follows:

Algorithm DP-TC-TC: An exact DP algorithm for $PDm|CO|\#(TC^A, TC^B)$

Input: m, $\mathcal{J}_i^X$, n_i^X, $p_{i,j}^X$ for $X = A, B$, $i = 1, 2, \ldots, m$ and $j = 1, 2, \ldots, n_i^X$

1 Renumber jobs in $\mathcal{J}_i^A$ and $\mathcal{J}_i^B$ in their respective SPT order, $i = 1, 2, \ldots, m$.
2 $\mathcal{L}(0, 0, 0) \leftarrow \{(0, 0)\}$, $\mathcal{L}(i, j_A, j_B) \leftarrow \emptyset$ for $i = 1, 2, \ldots, m$, $j_A = 0, 1, \ldots, n_i^A$, $j_B = 0, 1, \ldots, n_i^B$.
3 **for** $i = 1$ **to** m **do**
4 $\mathcal{L}(i, 0, 0) \leftarrow \mathcal{L}(i-1, n_{i-1}^A, n_{i-1}^B)$
5 **for** $j_A = 0$ **to** n_i^A **do**
6 **for** $j_B = 0$ **to** n_i^B **do**
7 **if** $j_A \geq 1$ **then**
8 **for** *each state* $(q_A, q_B) \in \mathcal{L}(i, j_A - 1, j_B)$ **do**
9 $\mathcal{L}(i, j_A, j_B) \leftarrow \mathcal{L}(i, j_A, j_B) \cup \{(q_A + P_{i,j_A}^A + P_{i,j_B}^B, q_B)\}$
10 **end**
11 **end**
12 **if** $j_B \geq 1$ **then**
13 **for** *each state* $(q_A, q_B) \in \mathcal{L}(i, j_A, j_B - 1)$ **do**
14 $\mathcal{L}(i, j_A, j_B) \leftarrow \mathcal{L}(i, j_A, j_B) \cup \{(q_A, q_B + P_{i,j_A}^A + P_{i,j_B}^B)\}$
15 **end**
16 **end**
17 Apply Rules R1 or R2.
18 **end**
19 **end**
20 **end**
21 Extract all PO points from $\mathcal{L}(m, n_m^A, n_m^B)$ to form set $\mathcal{X}$ of PO frontier, and generate a PO schedule $\sigma(q_A, q_B)$ via backtracking for each $(q_A, q_B) \in \mathcal{X}$.

Output: $\mathcal{X}$ and $\sigma(q_A, q_B)$ for each $(q_A, q_B) \in \mathcal{X}$

Theorem 4.1. *Algorithm* ***DP-TC-TC*** *solves problem* $PDm|CO|\#(\mathrm{TC}^A, \mathrm{TC}^B)$ *in* $O(n_A n_B \min\{\Delta_A, \Delta_B\})$ *time.*

Proof. The validity of Algorithm **DP-TC-TC** stems from two key facts: (i) its generation of all feasible states that constitute optimal schedules adherent to the properties outlined in Lemma 3.1, and (ii) throughout the elimination process, it preserves exclusively non-dominated states, as established by Lemma 4.1. The renumbering operation requires $\sum_{i=1}^m O(n_i^A \log n_i^A) + \sum_{i=1}^m O(n_i^B \log n_i^B) = O(n_A \log n_A + n_B \log n_B)$ time (line 1). The initialization takes $\sum_{i=1}^m (n_i^A n_i^B + 1) = O(n_A n_B)$ time (line 2). During the generation process

(lines 3-20), at most $O(\min\{\Delta_A, \Delta_B\})$ distinct states are retained in $\mathcal{L}(i, j_A, j_B)$ according to the elimination operation (line 17), and each state generates up to two new states. Hence, generating each $\mathcal{L}(i, j_A, j_B)$ can be accomplished in $O(\min\{\Delta_A, \Delta_B\})$ time. Consequently, the entire generation process takes $\sum_{i=1}^{m} O(n_i^A n_i^B \min\{\Delta_A, \Delta_B\}) = O(n_A n_B \min\{\Delta_A, \Delta_B\})$ time, which dominates the time required for extraction and backtracking (line 21). Therefore, the total running time is $O(n_A n_B \min\{\Delta_A, \Delta_B\})$. □

4.2 Problem $PDm|CO|\#(\mathrm{TC}^A, \mathrm{TG}^B)$

Given the results in Lemmas 3.1-3.3, we focus on schedules for problem $PDm|CO|\#(\mathrm{TC}^A, \mathrm{TG}^B)$ that obey the following constraints for each machine M_i $(i = 1, 2, \ldots, m)$: (i) the jobs in $\mathcal{J}_i^A$ are processed according to their SPT order; (ii) the valid jobs in $\mathcal{J}_i^B$ are processed according to their EDD order; and (iii) the invalid jobs in $\mathcal{J}_i^B$ are processed after all valid jobs in an arbitrary order.

Our DP algorithm consists of m stages, each corresponding to a distinct machine, with $n_i^A n_i^B + 1$ iterations conducted during the i-th stage. Specifically, in each stage-iteration denoted as (i, j_A, j_B), where $i = 1, 2, \ldots, m$, $j_A = 0, 1, \ldots, n_i^A$, and $j_B = 0, 1, \ldots, n_i^B$, a state space $\mathcal{L}(i, j_A, j_B)$ is generated. Each state in $\mathcal{L}(i, j_A, j_B)$ is a 3-tuple (t, q_A, q_B), representing a feasible schedule for jobs in $\mathcal{N}_{i-1} \cup \mathcal{J}_{i,j_A}^A \cup \mathcal{J}_{i,j_B}^B$. Here, t means the total processing time of valid jobs in $\mathcal{J}_{i,j_B}^B$, q_A denotes agent A's total completion time of the partial schedule, and q_B indicates agent B's objective value of the partial schedule. Note that the completion time of the last valid job on M_i is $t + P_{i,j_A}^A$. In the dynamic update step, $\mathcal{L}(i, j_A, j_B)$ is generated from the previous sets $\mathcal{L}(i, j_A - 1, j_B)$ and $\mathcal{L}(i, j_A, j_B - 1)$.

The DP algorithm is initialized by setting $\mathcal{L}(0, 0, 0) = \{(0, 0, 0)\}$. Let (t, q_A, q_B) be a given state in $\mathcal{L}(i, j_A, j_B)$. If $j_A \leq n_i^A - 1$ and $j_B \leq n_i^B - 1$, then three possible cases should be considered to generate a new state:

- **Case 1.** J_{i,j_A+1}^A is the next scheduled job on M_i. In this context, the completion time of J_{i,j_A+1}^A on M_i is equal to $t + P_{i,j_A+1}^A$, so its contribution to agent A's objective value is $t + P_{i,j_A+1}^A$. Hence, $(t, q_A + t + P_{i,j_A+1}^A, q_B)$ is added to $\mathcal{L}(i, j_A + 1, j_B)$.

- **Case 2.** J_{i,j_B+1}^B is the next scheduled job on M_i and it is valid. In this context, the completion time of J_{i,j_B+1}^A on M_i is equal to $t + P_{i,j_A}^A + p_{i,j_B+1}^B$. Observe that this case occurs only if (i) $t + P_{i,j_A}^A + p_{i,j_B+1}^B \leq d_{i,j_B+1}^B$ when $f^B = \mathrm{TWU}^B$, and (ii) $t + P_{i,j_A}^A + p_{i,j_B+1}^B < d_{i,j_B+1}^B + p_{i,j_B+1}^B$ (i.e., $t + P_{i,j_A}^A \leq d_{i,j_B+1}^B - 1$) when $f^B = \mathrm{TY}^B$. In this context, $(t + p_{i,j_B+1}^B, q_A, q_B)$ is added to $\mathcal{L}(i, j_A, j_B + 1)$ when $f^B = \mathrm{TWU}^B$, and $(t + p_{i,j_B+1}^B, q_A, q_B + \max\{0, t + P_{i,j_A}^A + p_{i,j_B+1}^B - d_{i,j_B+1}^B\})$ is added to $\mathcal{L}(i, j_A, j_B + 1)$ when $f^B = \mathrm{TY}^B$.

- **Case 3.** J_{i,j_B+1}^B is the next scheduled job on M_i and it is invalid. In this context, $(t, q_A, q_B + w_{i,j_B+1}^B)$ is added to $\mathcal{L}(i, j_A, j_B + 1)$ when $f^B = \mathrm{TWU}^B$, and $(t, q_A, q_B + p_{i,j_B+1}^B)$

is added to $\mathcal{L}(i, j_A, j_B + 1)$ when $f^B = \mathrm{TY}^B$.

To streamline the presentation of Cases 2 and 3, in a given schedule π, we introduce three functions $\varphi(i, j, X)$, $\psi(i, j, X, C)$, and $\omega(i, j, X)$ related to job $J_{i,j}^X$, where $X \in \{A, B\}$ and C denotes its completion time. These functions capture the differences in constraint conditions and penalty costs for the criteria TWU^X and TY^X. Define

$$\varphi(i, j, X) = \begin{cases} d_{i,j}^X, & \text{if } \mathrm{TG}^X = \mathrm{TWU}^X; \\ d_{i,j}^X + p_{i,j}^X - 1, & \text{if } \mathrm{TG}^X = \mathrm{TY}^X. \end{cases} \tag{3}$$

$$\psi(i, j, X, C) = \begin{cases} 0, & \text{if } \mathrm{TG}^X = \mathrm{TWU}^X; \\ \max\{0, C - d_{i,j}^X\}, & \text{if } \mathrm{TG}^X = \mathrm{TY}^X. \end{cases} \tag{4}$$

and

$$\omega(i, j, X) = \begin{cases} w_{i,j}^X, & \text{if } \mathrm{TG}^X = \mathrm{TWU}^X; \\ p_{i,j}^X, & \text{if } \mathrm{TG}^X = \mathrm{TY}^X. \end{cases} \tag{5}$$

For a given schedule π and $\mathrm{TG}^X \in \{\mathrm{TWU}^X, \mathrm{TY}^X\}$, functions (3)-(5) imply that:

- **(i)** $J_{i,j}^X$ is valid if and only if $C_{i,j}^X(\pi) \leq \varphi(i, j, X)$.
- **(ii)** a valid job $J_{i,j}^X$ contributes $\psi(i, j, X, C_{i,j}^X(\pi))$ to agent X's objective value.
- **(iii)** an invalid job $J_{i,j}^X$ contributes $\omega(i, j, X)$ to agent X's objective value.

Note that once $\mathcal{L}(i, n_i^A, n_i^B)$ has been constructed and $i \leq m-1$, the i-th stage terminates, after which the algorithm proceeds to the $(i+1)$-th stage. This initialization involves setting $\mathcal{L}(i + 1, 0, 0) = \{(0, q_A, q_B) : (t, q_A, q_B) \in \mathcal{L}(i, n_i^A, n_i^B)\}$. After completing all m stages, we can extract all PO points from $\mathcal{L}(m, n_m^A, n_m^B)$.

Moreover, we can apply the following elimination rule, which is straightforward to prove and will be employed to reduce the state space $\mathcal{L}(i, j_A, j_B)$.

Lemma 4.2. *For any two states (t, q_A, q_B) and (t, q'_A, q_B) in $\mathcal{L}(i, j_A, j_B)$ satisfying $q_A \leq q'_A$, the second one can be deleted.*

Based on the preceding analysis, we formulate the optimization algorithm (called **DP-TC-TG**) to solve problem $PDm|CO\#(\mathrm{TC}^A, \mathrm{TG}^B)$ as follows:

Theorem 4.2. *Algorithm **DP-TC-TG** solves problem $PDm|CO|\#(\mathrm{TC}^A, \mathrm{TG}^B)$ in $O(n_A n_B P_{\max}^B \Delta_B)$ time.*

Algorithm DP-TC-TG: An exact DP algorithm for $PDm|CO|\#(TC^A, \mathrm{TG}^B)$

Input: m, $\mathcal{J}_i^X$, n_i^X, $p_{i,j}^X$, $d_{i,j}^X$, $w_{i,j}^X$ for $X = A, B$, $i = 1, 2, \ldots, m$ and $j = 1, 2, \ldots, n_i^X$

Renumber jobs in $\mathcal{J}_i^A$ in the SPT order and jobs in $\mathcal{J}_i^B$ in the EDD order, $i = 1, 2, \ldots, m$.
$\mathcal{L}(0,0,0) \leftarrow \{(0,0,0)\}$, $\mathcal{L}(i, j_A, j_B) \leftarrow \emptyset$ for $i = 1, 2, \ldots, m$, $j_A = 0, 1, \ldots, n_i^A$, $j_B = 0, 1, \ldots, n_i^B$.
for $i = 1$ **to** m **do**
  $\mathcal{L}(i,0,0) \leftarrow \{(0, q_A, q_B) : (t, q_A, q_B) \in \mathcal{L}(i-1, n_{i-1}^A, n_{i-1}^B)\}$
  for $j_A = 0$ **to** n_i^A **do**
    for $j_B = 0$ **to** n_i^B **do**
      if $j_A \geq 1$ **then**
        for *each state* $(t, q_A, q_B) \in \mathcal{L}(i, j_A - 1, j_B)$ **do**
          $\mathcal{L}(i, j_A, j_B) \leftarrow \mathcal{L}(i, j_A, j_B) \cup \{(t, q_A + t + P_{i,j_A}^A, q_B)\}$
        end
      end
      if $j_B \geq 1$ **then**
        for *each state* $(t, q_A, q_B) \in \mathcal{L}(i, j_A, j_B - 1)$ **do**
          if $t + P_{i,j_A}^A + p_{i,j_B}^B \leq \varphi(i, j_B, B)$ **then**
            $\mathcal{L}(i, j_A, j_B) \leftarrow \mathcal{L}(i, j_A, j_B) \cup \{(t + p_{i,j_B}^B, q_A, q_B + \psi(i, j_B, B, t + P_{i,j_A}^A + p_{i,j_B}^B)), (t, q_A, q_B + \omega(i, j_B, B))\}$
          end
          else
            $\mathcal{L}(i, j_A, j_B) \leftarrow \mathcal{L}(i, j_A, j_B) \cup \{(t, q_A, q_B + \omega(i, j_B, B)\}$
          end
        end
      end
      Among all elements (t, q_A, q_B) in $\mathcal{L}(i, j_A, j_B)$ sharing identical t and q_B values, retain only the element with minimal q_A value.
    end
  end
end
Extract all PO points from $\mathcal{L}(m, n_m^A, n_m^B)$ to form set $\mathcal{X}$ of PO frontier, and generate a PO schedule $\sigma(q_A, q_B)$ via backtracking for each $(q_A, q_B) \in \mathcal{X}$.

Output: $\mathcal{X}$ and $\sigma(q_A, q_B)$ for each $(q_A, q_B) \in \mathcal{X}$

Proof. The validity of Algorithm **DP-TC-TG** stems from two key facts: (i) its generation of all feasible states that constitute optimal schedules adherent to the properties outlined in Lemmas 3.1-3.3, and (ii) throughout the elimination process, it preserves exclusively non-dominated states, as established by Lemma 4.2. The renumbering operation requires $O(n_A \log n_A + n_B \log n_B)$ time (line 1). The initialization takes $O(n_A n_B)$ time (line 2). During the generation process (lines 3-25), due to the bounds on $t \leq P_i^B$ and $q_B \leq \Delta_B$, after the elimination operation (line 22), at most $O(P_i^B \Delta_B)$ distinct states are retained in $\mathcal{L}(i, j_A, j_B)$, and each state generates up to three new states. Hence, generating each $\mathcal{L}(i, j_A, j_B)$ can be accomplished in $O(P_i^B \Delta_B)$ time. Consequently, the entire generation process takes $\sum_{i=1}^{m} O(n_i^A n_i^B P_i^B \Delta_B) \leq O(n_A n_B P_{\max}^B \Delta_B)$ time, which dominates the time required for extraction and backtracking (line 26). Therefore, the total running time is $O(n_A n_B P_{\max}^B \Delta_B\})$. □

From the definition of Δ_B given in (2), Theorem 4.2 leads to the following three results.

Corollary 4.1. *Problem* $PDm|CO|\#(\mathrm{TC}^A, \mathrm{TY}^B)$ *can be solved in* $O(n_A n_B P_{\max}^B P(B))$ *time.*

Corollary 4.2. *Problem* $PDm|CO|\#(\mathrm{TC}^A, \mathrm{TWU}^B)$ *can be solved in* $O(n_A n_B P_{\max}^B W(B))$ *time.*

Corollary 4.3. *Problem* $PDm|CO|\#(\mathrm{TC}^A, \mathrm{TU}^B)$ *can be solved in* $O(n_A n_B^2 P_{\max}^B)$ *time.*

4.3 Problem $PDm|CO|\#(\mathrm{TG}^A, \mathrm{TG}^B)$

Given the results in Lemmas 3.2 and 3.3, we focus on schedules for problem $PDm|CO|\#(\mathrm{TG}^A, \mathrm{TG}^B)$ that obey the following constraints for each machine M_i ($i = 1, 2, \ldots, m$): (i) the valid jobs in $\mathcal{J}_i^A$ are processed according to their EDD order; (ii) the valid jobs in $\mathcal{J}_i^B$ are also processed according to their EDD order; and (iii) the invalid jobs in $\mathcal{J}_i$ are processed after all valid jobs in an arbitrary order.

Our DP algorithm consists of m stages, each corresponding to a distinct machine, with $n_i^A n_i^B + 1$ iterations conducted during the i-th stage. Specifically, in each stage-iteration denoted as (i, j_A, j_B), where $i = 1, 2, \ldots, m$, $j_A = 0, 1, \ldots, n_i^A$, and $j_B = 0, 1, \ldots, n_i^B$, a state space $\mathcal{L}(i, j_A, j_B)$ is generated. Each state in $\mathcal{L}(i, j_A, j_B)$ is a 3-tuple (t, q_A, q_B), representing a feasible schedule for jobs in $\mathcal{N}_{i-1} \cup \mathcal{J}_{i,j_A}^A \cup \mathcal{J}_{i,j_B}^B$. Here, t means the completion time of the last valid job in $\mathcal{J}_{i,j_A}^A \cup \mathcal{J}_{i,j_B}^B$, q_A denotes agent A's objective value of the partial schedule, and q_B indicates agent B's objective value of the partial schedule. In the dynamic update step, $\mathcal{L}(i, j_A, j_B)$ is generated from the previous sets $\mathcal{L}(i, j_A - 1, j_B)$ and $\mathcal{L}(i, j_A, j_B - 1)$.

The DP algorithm is initialized by setting $\mathcal{L}(0, 0, 0) = \{(0, 0, 0)\}$. Let (t, q_A, q_B) be a given state in $\mathcal{L}(i, j_A, j_B)$. If $j_A \leq n_i^A - 1$ and $j_B \leq n_i^B - 1$, then four possible cases should be considered to generate a new state:

- **Case 1.** J^A_{i,j_A+1} is the next scheduled job on M_i and it is valid. In this context, the completion time of J^A_{i,j_A+1} on M_i is equal to $t + p^A_{i,j_A+1}$, and its contribution to agent A's objective value is $\psi(i, j_A + 1, A, t + p^A_{i,j_A+1})$. Hence, $(t + p^A_{i,j_A+1}, q_A + \psi(i, j_A + 1, A, t + p^A_{i,j_A+1}), q_B)$ is added to $\mathcal{L}(i, j_A + 1, j_B)$. Observe that this case occurs only when $t + p^A_{i,j_A+1} \leq \varphi(i, j_A + 1, A)$. Here, $\psi(i, \cdot, \cdot)$ and $\varphi(i, \cdot, \cdot)$ are defined by (3) and (4), respectively.

- **Case 2.** J^A_{i,j_A+1} is the next scheduled job on M_i and it is invalid. In this context, it contributes $\omega(i, j_A+1, A)$ to agent A's objective value. Hence, $(t, q_A+\omega(i, j_A+1, A), q_B)$ is added to $\mathcal{L}(i, j_A + 1, j_B)$, where $\omega(i, \cdot, \cdot)$ is defined by (5).

- **Case 3.** J^A_{i,j_B+1} is the next scheduled job on M_i and it is valid. Similar to Case 1, $(t+p^B_{i,j_B+1}, q_A, q_B+\psi(i, j_B+1, B, t+p^B_{i,j_B+1})$ is added to $\mathcal{L}(i, j_A, j_B+1)$. Observe that this case occurs only when $t + p^B_{i,j_B+1} \leq \varphi(i, j_B + 1, B)$.

- **Case 4.** J^B_{i,j_B+1} is the next scheduled job on M_i and it is invalid. Similar to Case 2, $(t, q_A, q_B + \omega(i, j_B + 1, B))$ is added to $\mathcal{L}(i, j_A, j_B + 1)$.

Note that once $\mathcal{L}(i, n^A_i, n^B_i)$ has been constructed and $i \leq m-1$, the i-th stage terminates, after which the algorithm proceeds to the $(i+1)$-th stage. This initialization involves setting $\mathcal{L}(i + 1, 0, 0) = \{(0, q_A, q_B) : (t, q_A, q_B) \in \mathcal{L}(i, n^A_i, n^B_i)\}$. After completing all m stages, we can extract all PO points from $\mathcal{L}(m, n^A_m, n^B_m)$.

Moreover, similar to Lemma 4.2, the following elimination rule is used to reduce the state space $\mathcal{L}(i, j_A, j_B)$.

Lemma 4.3. *For any two states (t, q_A, q_B) and (t', q_A, q_B) in $\mathcal{L}(i, j_A, j_B)$ satisfying $t \leq t'$, the second one can be deleted.*

Based on the preceding analysis, we formulate the optimization algorithm (called **DP-TG-TG**) to solve problem $PDm|CO\#(\mathrm{TG}^A, \mathrm{TG}^B)$ as follows:

Theorem 4.3. *Algorithm **DP-TG-TG** solves problem $PDm|CO|\#(\mathrm{TG}^A, \mathrm{TG}^B)$ in $O(n_A n_B \Delta_A \Delta_B)$ time.*

Proof. The validity of Algorithm **DP-TG-TG** stems from two key facts: (i) its generation of all feasible states that constitute optimal schedules adherent to the properties outlined in Lemmas 3.2-3.3, and (ii) throughout the elimination process, it preserves exclusively non-dominated states, as established by Lemma 4.3. The renumbering operation requires $O(n_A \log n_A + n_B \log n_B)$ time (line 1). The initialization takes $O(n_A n_B)$ time (line 2). During the generation process (lines 3-30), due to the bounds on $q_A \leq \Delta_A$, and $q_B \leq \Delta_B$, after the elimination operation (line 27), at most $O(\Delta_A \Delta_B)$ distinct states are retained in $\mathcal{L}(i, j_A, j_B)$, and each state generates up to four new states. Hence, generating each $\mathcal{L}(i, j_A, j_B)$ can be accomplished in $O(\Delta_A \Delta_B)$ time. Consequently, the entire generation

Algorithm DP-TG-TG: An exact DP algorithm for $PDm|CO|\#(\mathrm{TG}^A, \mathrm{TG}^B)$

Input: m, $\mathcal{J}_i^X$, n_i^X, $p_{i,j}^X$, $d_{i,j}^X$, $w_{i,j}^X$ for $X = A, B$, $i = 1, 2, \ldots, m$ and $j = 1, 2, \ldots, n_i^X$

Renumber jobs in $\mathcal{J}_i^A$ and $\mathcal{J}_i^B$ in their respective EDD order, $i = 1, 2, \ldots, m$.

$\mathcal{L}(0,0,0) \leftarrow \{(0,0,0)\}$, $\mathcal{L}(i, j_A, j_B) \leftarrow \emptyset$ for $i = 1, 2, \ldots, m$, $j_A = 0, 1, \ldots, n_i^A$, $j_B = 0, 1, \ldots, n_i^B$.

for $i = 1$ **to** m **do**

  $\mathcal{L}(i, 0, 0) \leftarrow \{(0, q_A, q_B) : (t, q_A, q_B) \in \mathcal{L}(i-1, n_{i-1}^A, n_{i-1}^B)\}$

  for $j_A = 0$ **to** n_i^A **do**

    for $j_B = 0$ **to** n_i^B **do**

      if $j_A \geq 1$ **then**

        for *each state* $(t, q_A, q_B) \in \mathcal{L}(i, j_A - 1, j_B)$ **do**

          if $t + p_{i,j_A}^A \leq \varphi(i, j_A, A)$ **then**

            $\mathcal{L}(i, j_A, j_B) \leftarrow \mathcal{L}(i, j_A, j_B) \cup \{(t + p_{i,j_A}^A, q_A + \psi(i, j_A, A, t + p_{i,j_A}^A), q_B), (t, q_A + \omega(i, j_A, A), q_B)\}$

          end

          else

            $\mathcal{L}(i, j_A, j_B) \leftarrow \mathcal{L}(i, j_A, j_B) \cup \{(t, q_A + \omega(i, j_A, A), q_B\}$

          end

        end

      end

      if $j_B \geq 1$ **then**

        for *each state* $(t, q_A, q_B) \in \mathcal{L}(i, j_A, j_B - 1)$ **do**

          if $t + p_{i,j_B}^B \leq \varphi(i, j_B, B)$ **then**

            $\mathcal{L}(i, j_A, j_B) \leftarrow \mathcal{L}(i, j_A, j_B) \cup \{(t + p_{i,j_B}^B, q_A, q_B + \psi(i, j_B, B, t + p_{i,j_B}^B)), (t, q_A, q_B + \omega(i, j_B, B))\}$

          end

          else

            $\mathcal{L}(i, j_A, j_B) \leftarrow \mathcal{L}(i, j_A, j_B) \cup \{(t, q_A, q_B + \omega(i, j_B, B)\}$

          end

        end

      end

      Among all elements (t, q_A, q_B) in $\mathcal{L}(i, j_A, j_B)$ sharing identical q_A and q_B values, retain only the element with minimal t value.

    end

  end

end

Extract all PO points from $\mathcal{L}(m, n_m^A, n_m^B)$ to form set $\mathcal{X}$ of PO frontier, and generate a PO schedule $\sigma(q_A, q_B)$ via backtracking for each $(q_A, q_B) \in \mathcal{X}$.

Output: $\mathcal{X}$ and $\sigma(q_A, q_B)$ for each $(q_A, q_B) \in \mathcal{X}$

process takes $\sum_{i=1}^{m} O(n_i^A n_i^B \Delta_A \Delta_B)$ time, which dominates the time required for extraction and backtracking (line 31). Therefore, the total running time is $O(n_A n_B \Delta_A \Delta_B)$. □

From the definitions of Δ_A and Δ_B given in (1) and (2), Theorem 4.3 leads to the following four results.

Corollary 4.4. *Problem* $PDm|CO|\#(\mathrm{TWU}^A, \mathrm{TWU}^B)$ *can be solved in* $O(n_A n_B W(A) W(B))$ *time.*

Corollary 4.5. *Problem* $PDm|CO|\#(\mathrm{TY}^A, \mathrm{TY}^B)$ *can be solved in* $O(n_A n_B P(A) P(B))$ *time.*

Corollary 4.6. *Problem* $PDm|CO|\#(\mathrm{TY}^A, \mathrm{TWU}^B)$ *can be solved in* $O(n_A n_B P(A) W(B))$ *time.*

Corollary 4.7. *Problem* $PDm|CO|\#(\mathrm{TY}^A, \mathrm{TU}^B)$ *can be solved in* $O(n_A n_B^2 P(A))$ *time.*

4.4 Problem $PDm|CO|\#(\mathrm{TWU}^A, \mathrm{TWU}^B)$

Although the unified algorithm **DP-TG-TG** (designed in Section 4.3) solves problem $PDm|CO|\#(\mathrm{TWU}^A, \mathrm{TWU}^B)$ in $O(n_A n_B W(A) W(B))$ time, we next propose an improved DP algorithm with $O(nW(A)W(B))$ time complexity.

Given the results in Lemma 3.4, we focus on schedules for problem $PDm|CO|\#(\mathrm{TWU}^A, \mathrm{TWU}^B)$ that obey the following constraints for each machine M_i $(i = 1, 2, \ldots, m)$: (i) the early jobs in $\mathcal{J}_i$ are processed according to their EDD order; and (ii) the tardy jobs in $\mathcal{J}_i$ are processed after all early jobs in an arbitrary order.

For convenience, we assume that jobs in $\mathcal{J}_i$ $(i = 1, 2, \ldots, m)$ are renumbered in the EDD order such that $d_{i,1} \leq d_{i,2} \leq \cdots \leq d_{i,n_i}$. Consequently, we use $J_{i,j}$, $d_{i,j}$, and $w_{i,j}$ to denote the j-th job in this sequence, its due date, and its weight, respectively.

Our DP algorithm consists of m stages, each corresponding to a distinct machine, with $n_i + 1$ iterations conducted during the i-th stage. Specifically, in each stage-iteration denoted as (i, j), where $i = 1, 2, \ldots, m$ and $j = 0, 1, \ldots, n_i$, a state space $\mathcal{L}(i, j)$ is generated. Each state in $\mathcal{L}(i, j)$ is a 3-tuple (t, q_A, q_B), representing a feasible schedule for jobs in $\mathcal{N}_{i-1} \cup \{J_{i,1}, J_{i,2}, \ldots, J_{i,j}\}$. Here, t means the completion time of the last early job in $\{J_{i,1}, J_{i,2}, \ldots, J_{i,j}\}$, q_A denotes agent A's objective value of the partial schedule, and q_B indicates agent B's objective value of the partial schedule. In the dynamic update step, $\mathcal{L}(i, j)$ is generated from the previous set $\mathcal{L}(i, j-1)$.

The DP algorithm is initialized by setting $\mathcal{L}(0, 0) = \{(0, 0, 0)\}$. Let (t, q_A, q_B) be a given state in $\mathcal{L}(i, j)$. If $j \leq n_i - 1$, then three possible cases should be considered to generate a new state:

- **Case 1.** $J_{i,j+1}$ is scheduled early on M_i. In this context, $(t+p_{i,j+1}, q_A, q_B)$ is added to $\mathcal{L}(i, j+1)$. Observe that this case occurs only when $t + p_{i,j} \leq d_{i,j}$.

- **Case 2.** $J_{i,j+1}$ is scheduled tardy on M_i and it comes from agent A. In this context, it contributes $w(i, j+1)$ to agent A's objective value. Hence, $(t, q_A + w_{i,j}, q_B)$ is added to $\mathcal{L}(i, j+1)$.

- **Case 3.** $J_{i,j+1}$ is scheduled tardy on M_i and it comes from agent B. Similar to Case 2, $(t, q_A, q_B + w_{i,j}))$ is added to $\mathcal{L}(i, j+1)$.

Note that once $\mathcal{L}(i, n_i)$ has been constructed and $i \leq m-1$, the i-th stage terminates, after which the algorithm proceeds to the $(i+1)$-th stage. This initialization involves setting $\mathcal{L}(i+1, 0) = \{(0, q_A, q_B) : (t, q_A, q_B) \in \mathcal{L}(i, n_i)\}$. After completing all m stages, we can extract all PO points from $\mathcal{L}(m, n_m)$.

Moreover, the following easy-to-prove elimination rule is used to reduce the state space $\mathcal{L}(i, j)$.

Lemma 4.4. *For any two states (t, q_A, q_B) and (t', q_A, q_B) in $\mathcal{L}(i, j)$ satisfying $t \leq t'$, the second one can be deleted.*

Based on the preceding analysis, we formulate the improved optimization algorithm (called **DP-I-TWU**) to solve problem $PDm|CO|\#(\mathrm{TWU}^A, \mathrm{TWU}^B)$ as follows:

Theorem 4.4. *Algorithm **DP-I-TWU** solves problem $PDm|CO|\#(\mathrm{TWU}^A, \mathrm{TWU}^B)$ in $O(nW(A)W(B))$ time.*

Proof. The validity of Algorithm **DP-I-TWU** stems from two key facts: (i) its generation of all feasible states that constitute optimal schedules adherent to the properties outlined in Lemma 3.4, and (ii) throughout the elimination process, it preserves exclusively non-dominated states, as established by Lemma 4.4. The renumbering operation requires $O(n_A \log n_A + n_B \log n_B)$ time (line 1). The initialization takes $\sum_{i=1}^{m} O(n_i) = O(n)$ time (line 2). During the generation process (lines 3-26), due to the bounds on $q_A \leq W(A)$ and $q_A \leq W(B)$, after the elimination operation (line 24), at most $O(W(A)W(B))$ distinct states are retained in $\mathcal{L}(i, j)$, and each state generates up to two new states. Hence, generating each $\mathcal{L}(i, j)$ can be accomplished in $O(W(A)W(B))$ time. Consequently, the entire generation process takes $\sum_{i=1}^{m} O(n_i W(A)W(B)) = O(nW(A)W(B))$ time, which dominates the time required for extraction and backtracking (line 27). Therefore, the total running time is $O(nW(A)W(B))$. □

Corollary 4.8. *Problem $PDm|CO|\#(\mathrm{TWU}^A, \mathrm{TU}^B)$ can be solved in $O(nn_B W(A))$ time.*

Corollary 4.9. *Problem $PDm|CO|\#(\mathrm{TU}^A, \mathrm{TU}^B)$ can be solved in $O(nn_A n_B)$ time.*

Algorithm DP-I-TWU: An exact DP algorithm for $PDm|CO|\#(\text{TWU}^A, \text{TWU}^B)$

Input: m, $\mathcal{J}_i$, n_i, $p_{i,j}^X$, $d_{i,j}^X$, $w_{i,j}^X$ for $X = A, B$, $i = 1, 2, \ldots, m$ and $j = 1, 2, \ldots, n_i^X$

Renumber jobs in $\mathcal{J}_i$ in the EDD order such that $d_{i,1} \leq d_{i,2} \leq \cdots \leq d_{i,n_i}$, $i = 1, 2, \ldots, m$.
$\mathcal{L}(0,0) \leftarrow \{(0,0,0)\}$, $\mathcal{L}(i,j) \leftarrow \emptyset$ for $i = 1, 2, \ldots, m$, $j = 0, 1, \ldots, n_i$
for $i = 1$ **to** m **do**
    $\mathcal{L}(i,0) \leftarrow \{(0, q_A, q_B) : (t, q_A, q_B) \in \mathcal{L}(i-1, n_{i-1})\}$
    for $j = 1$ **to** n_i **do**
        for *each state* $(t, q_A, q_B) \in \mathcal{L}(i, j-1)$ **do**
            if $J_{i,j} \in \mathcal{J}_i^A$ **then**
                if $t + p_{i,j} \leq d_{i,j}$ **then**
                    $\mathcal{L}(i,j) \leftarrow \mathcal{L}(i,j) \cup \{(t + p_{i,j}, q_A, q_B), (t, q_A + w_{i,j}, q_B)\}$
                end
                else
                    $\mathcal{L}(i,j) \leftarrow \mathcal{L}(i,j) \cup \{(t, q_A + w_{i,j}, q_B)\}$
                end
            end
            if $J_{i,j} \in \mathcal{J}_i^B$ **then**
                if $t + p_{i,j} \leq d_{i,j}$ **then**
                    $\mathcal{L}(i,j) \leftarrow \mathcal{L}(i,j) \cup \{(t + p_{i,j}, q_A, q_B), (t, q_A, q_B + w_{i,j})\}$
                end
                else
                    $\mathcal{L}(i,j) \leftarrow \mathcal{L}(i,j) \cup \{(t, q_A, q_B + w_{i,j})\}$
                end
            end
        end
        Among all elements in $\mathcal{L}(i,j)$ sharing identical q_A and q_B values, retain only the element with minimal t value.
    end
end
Extract all PO points from $\mathcal{L}(m, n_m)$ to form set $\mathcal{X}$ of PO frontier, and generate a PO schedule $\sigma(q_A, q_B)$ via backtracking for each $(q_A, q_B) \in \mathcal{X}$.

Output: $\mathcal{X}$ and $\sigma(q_A, q_B)$ for each $(q_A, q_B) \in \mathcal{X}$

5 Approximation schemes

In this section, we design approximate PO frontiers for the six problems studied in Section 4. An algorithm for problem $PDm|CO|\#(f^A, f^B)$ is said to be a (ρ_A, ρ_B)-*approximation* if it provides a (ρ_A, ρ_B)-approximate PO frontier. A family of algorithms that achieves $(1+\epsilon, 1+\epsilon)$-approximations for every $\epsilon > 0$ is classified as a *fully polynomial time approximation scheme* (FPTAS) if its time complexity is polynomial in both the input size and $1/\epsilon$.

5.1 Problem $PDm|CO|\#(\mathrm{TC}^A, \mathrm{TC}^B)$

By applying the *trimming-the-state-space* technique (Ibarra and Kim, 1975), we transform the exact DP algorithm designed in Section 4.1 for problem $PDm|CO|\#(\mathrm{TC}^A, \mathrm{TC}^B)$ into an FPTAS. This algorithm delivers a $(1, 1+\epsilon)$-approximate PO frontier. The transformation is realized by partitioning the range of one agent's objective function values into geometrically increasing intervals, a common approach in constructing approximation schemes (Li and Yuan, 2020; Boeckmann et al., 2023).

Algorithm DP-TC-TC(ϵ): An FPTAS for $PDm|CO|\#(TC^A, TC^B)$

Input: m, n, ϵ, $\mathcal{J}_i^X$, n_i^X, $p_{i,j}^X$ for $X = A, B$, $i = 1, 2, \ldots, m$ and $j = 1, 2, \ldots, n_i^X$

1 Set $\delta = (1+\epsilon)^{\frac{1}{n-1}}$ and $\Delta_B = \sum_{i=1}^{m}(n_i^B P_i^A + \sum_{j=1}^{n_i^B}(n_i^B - j + 1)p_{i,j}^B)$.

2 Partition $[0, \Delta_B]$ into $\eta_B + 1$ subintervals $I_0 = [0, 1)$, $I_1 = [1, \delta)$, $I_2 = [\delta, \delta^2)$, $\cdots$, $I_{\eta_B - 1} = [\delta^{\eta_B - 2}, \delta^{\eta_B - 1})$, $I_{\eta_B} = [\delta^{\eta_B - 1}, \Delta_B]$, where $\eta_B = \lceil \log_\delta \Delta_B \rceil = O(\frac{n}{\epsilon} \log \Delta_B)$.

3 Renumber jobs in $\mathcal{J}_i^A$ and $\mathcal{J}_i^B$ in their respective SPT order, $i = 1, 2, \ldots, m$.

4 $\mathcal{L}_\epsilon(0, 0, 0) \leftarrow \{(0, 0)\}$, $\mathcal{L}_\epsilon(i, j_A, j_B) \leftarrow \emptyset$ for $i = 1, 2, \ldots, m$, $j_A = 0, 1, \ldots, n_i^A$, $j_B = 0, 1, \ldots, n_i^B$.

5 **for** $i = 1$ **to** m **do**

6 $\quad \mathcal{L}_\epsilon(i, 0, 0) \leftarrow \mathcal{L}_\epsilon(i-1, n_{i-1}^A, n_{i-1}^B)$

7 $\quad$ **for** $j_A = 0$ **to** n_i^A **do**

8 $\quad\quad$ **for** $j_B = 0$ **to** n_i^B **do**

9 $\quad\quad\quad$ Run lines 7-16 in Algorithm **DP-TC-TC** to generate $\mathcal{L}_\epsilon(i, j_A, j_B)$.

10 $\quad\quad\quad$ Among all elements (q_A, q_B) in $\mathcal{L}_\epsilon(i, j_A, j_B)$ whose q_B value falls into the same subinterval I_r, retain only the element with minimal q_A value.

11 $\quad\quad$ **end**

12 $\quad$ **end**

13 **end**

14 Extract all PO points from $\mathcal{L}_\epsilon(m, n_m^A, n_m^B)$ to form set $\mathcal{X}_\epsilon$, and generate a corresponding schedule $\sigma(q_A, q_B)$ via backtracking for each $(q_A, q_B) \in \mathcal{X}_\epsilon$.

Output: $\mathcal{X}_\epsilon$ and $\sigma(q_A, q_B)$ for each $(q_A, q_B) \in \mathcal{X}_\epsilon$

Lemma 5.1. *For every state $(q_A, q_B) \in \mathcal{L}(i, j_A, j_B)$ generated by Algorithm **DP-TC-TC**, there exists a state $(\widetilde{q_A}, \widetilde{q_B}) \in \mathcal{L}_\epsilon(i, j_A, j_B)$ with $\widetilde{q_A} \le q_A$ and $\widetilde{q_B} \le \delta^{j-1} q_B$ in Algorithm **DP-TC-TC(ϵ)**, where $j = N_{i-1} + j_A + j_B$.*

Proof. We prove the statement by induction on j from 1 to n. For the base case $j = 1$, either $J_{1,1}^A$ or $J_{1,1}^B$ is the first scheduled job on M_1 in both Algorithms **DP-TC-TC** and **DP-TC-TC(ϵ)**. From the execution of these two algorithms, we have $\mathcal{L}_\epsilon(1, 1, 0) = \mathcal{L}(1, 1, 0) = \{(p_{1,1}^A, 0)\}$ and $\mathcal{L}_\epsilon(1, 0, 1) = \mathcal{L}(1, 0, 1) = \{(0, p_{1,1}^B)\}$. Thus, the statement is true for $j = 1$.

Now consider the scenario where $j \ge 2$ jobs has been processed during the algorithm and assume that the statement holds for $j - 1$. Note that for a given $j \ge 2$, there exists a

unique machine index $i \in \{1, 2, \ldots, m\}$ such that $N_{i-1} < j \leq N_i$. Let $(q_A, q_B) \in \mathcal{L}(i, j_A, j_B)$ be a state generated by Algorithm **DP-TC-TC** for jobs in $\mathcal{N}_{i-1} \cup \mathcal{J}^A_{i,j_A} \cup \mathcal{J}^B_{i,j_B}$, where $j = N_{i-1} + j_A + j_B$. The state $(q_A, q_B) \in \mathcal{L}(i, j_A, j_B)$ may be generated in Algorithm **DP-TC-TC** through two distinct cases.

Case 1. J^A_{i,j_A} is the last scheduled job on M_i (assuming $j_A \geq 1$). In this context, according to the execution of Algorithm **DP-TC-TC**, there exists a state $(q'_A, q_B) \in \mathcal{L}(i, j_A - 1, j_B)$, where $q'_A = q_A - P^A_{i,j_A} - P^B_{i,j_B}$. Based on the induction hypothesis, there exists a state $(\overline{q'_A}, \overline{q_B}) \in \mathcal{L}_\epsilon(i, j_A - 1, j_B)$ with $\overline{q'_A} \leq q'_A$ and $\overline{q_B} \leq \delta^{j-2} q_B$ in Algorithm **DP-TC-TC(ϵ)**. Since $j_A \geq 1$, the state $(\overline{q'_A} + P^A_{i,j_A} + P^B_{i,j_B}, \overline{q_B})$ is added to $\mathcal{L}_\epsilon(i, j_A, j_B)$. However, this state may be eliminated in Algorithm **DP-TC-TC(ϵ)** (line 10) by another state $(\widetilde{q_A}, \widetilde{q_B}) \in \mathcal{L}_\epsilon(i, j_A, j_B)$. For the state $(\widetilde{q_A}, \widetilde{q_B})$, according to the execution of Algorithm **DP-TC-TC(ϵ)**, we get $\widetilde{q_A} \leq \overline{q'_A} + P^A_{i,j_A} + P^B_{i,j_B} \leq q'_A + P^A_{i,j_A} + P^B_{i,j_B} = q_A$ and $\widetilde{q_B} \leq \delta \overline{q_B} \leq \delta^{j-1} q_B$. This establishes that the statement is true when J^A_{i,j_A} is scheduled last on M_i.

Case 2. J^B_{i,j_B} is the last scheduled job on M_i (assuming $j_B \geq 1$). In this context, according to the execution of Algorithm **DP-TC-TC**, there exists a state $(q_A, q'_B) \in \mathcal{L}(i, j_A, j_B - 1)$, where $q'_B = q_B - P^A_{i,j_A} - P^B_{i,j_B}$. Based on the induction hypothesis, there exists a state $(\overline{q_A}, \overline{q'_B}) \in \mathcal{L}_\epsilon(i, j_A, j_B - 1)$ with $\overline{q_A} \leq q_A$ and $\overline{q'_B} \leq \delta^{j-2} q'_B$ in Algorithm **DP-TC-TC(ϵ)**. Since $j_B \geq 1$, the state $(\overline{q_A}, \overline{q'_B} + P^A_{i,j_A} + P^B_{i,j_B})$ is added to $\mathcal{L}_\epsilon(i, j_A, j_B)$. However, this state may be eliminated in Algorithm **DP-TC-TC(ϵ)** (line 10) by another state $(\widetilde{q_A}, \widetilde{q_B}) \in \mathcal{L}_\epsilon(i, j_A, j_B)$. For the state $(\widetilde{q_A}, \widetilde{q_B})$, according to the execution of Algorithm **DP-TC-TC(ϵ)**, we get $\widetilde{q_A} \leq \overline{q_A} \leq q_A$ and $\widetilde{q_B} \leq \delta(\overline{q'_B} + P^A_{i,j_A} + P^B_{i,j_B}) \leq \delta(\delta^{j-2} q'_B + P^A_{i,j_A} + P^B_{i,j_B}) < \delta^{j-1}(q'_B + P^A_{i,j_A} + P^B_{i,j_B}) = q_B$. This establishes that the statement is true when J^B_{i,j_B} is scheduled last on M_i.

Thus, the induction hypothesis in both cases. □

Theorem 5.1. *Algorithm **DP-TC-TC(ϵ)** is an FPTAS for problem $PDm|CO|\#(\mathrm{TC}^A, \mathrm{TC}^B)$, which constructs a $(1, 1+\epsilon)$-approximate PO frontier in $O(\frac{n n_A n_B \log \Delta_B}{\epsilon})$ time.*

Proof. For every PO point $(q_A, q_B) \in \mathcal{X} \subseteq \mathcal{L}(m, n^A_m, n^B_m)$, utilizing the result of Lemma 5.1 and noting that $n = N_{m-1} + n^A_m + n^B_m$, Algorithm **DP-TC-TC($\epsilon$)** generates a state $(\widetilde{q_A}, \widetilde{q_B}) \in \mathcal{L}_\epsilon(m, n^A_m, n^B_m)$ such that $\widetilde{q_A} \leq q_A$ and $\widetilde{q_B} \leq \delta^{n-1} q_B$. Since $\delta = (1+\epsilon)^{\frac{1}{n-1}}$, we have $\widetilde{q_B} \leq (1+\epsilon) q_B$. Base on the execution of line 14 in Algorithm **DP-TC-TC(ϵ)**, we can find a state $(\widehat{q_A}, \widehat{q_B}) \in \mathcal{X}_\epsilon$ with $\widehat{q_A} \leq \widetilde{q_A}$ and $\widehat{q_B} \leq (1+\epsilon)\widetilde{q_A}$. Consequently, Algorithm **DP-TC-TC(ϵ)** constructs a $(1, 1+\epsilon)$-approximate PO frontier for problem $PDm|CO|\#(\mathrm{TC}^A, \mathrm{TC}^B)$.

Next, the running time of Algorithm **DP-TC-TC(ϵ)** is estimated. The interval partition takes $O(\frac{n \log \Delta_B}{\epsilon})$ time (lines 1-2). The renumbering operation requires $O(n_A \log n_A + n_B \log n_B)$ time (line 3). The initialization takes $O(n_A n_B)$ time (line 4). During the generation process (lines 5-13), at most $O(\frac{n \log \Delta_B}{\epsilon})$ distinct states are retained in $\mathcal{L}_\epsilon(i, j_A, j_B)$ due to the elimination operation (line 10), and each state generates up to two new states. Hence, generating each $\mathcal{L}_\epsilon(i, j_A, j_B)$ can be accomplished in $O(\frac{n \log \Delta_B}{\epsilon})$ time. Consequently, the en-

tire generation process takes $\sum_{i=1}^{m} O(\frac{nn_i^A n_i^B \log \Delta_B}{\epsilon}) = O(\frac{nn_A n_B \log \Delta_B}{\epsilon})$ time, which dominates the time required for extraction and backtracking (line 14). Therefore, the total running time is $O(\frac{nn_A n_B \log \Delta_B}{\epsilon})$. □

Remark 5.1. *Swapping the roles of agents A and B in Algorithm **DP-TC-TC(ϵ)** also yields an FPTAS for problem $PDm|CO|\#(\mathrm{TC}^A, \mathrm{TC}^B)$, constructing a $(1+\epsilon, 1)$-approximate PO frontier in $O(\frac{nn_A n_B \log \Delta_A}{\epsilon})$ time.*

5.2 Problem $PDm|CO|\#(\mathrm{TC}^A, \mathrm{TG}^B)$

Following a similar approach as outlined in Section 5.1, we can transform the exact DP algorithm designed in Section 4.2 for problem $PDm|CO|\#(\mathrm{TC}^A, \mathrm{TG}^B)$ into an FPTAS. This modified algorithm delivers a $(1+\epsilon, 1+\epsilon)$-approximate PO frontier.

Algorithm DP-TC-TG(ϵ): An FPTAS for $PDm|CO|\#(TC^A, TG^B)$

Input: m, n, ϵ, $\mathcal{J}_i^X$, n_i^X, $p_{i,j}^X$, $d_{i,j}^X$, $w_{i,j}^X$ for $X = A, B$, $i = 1, 2, \ldots, m$ and $j = 1, 2, \ldots, n_i^X$

1 Set $\delta = (1+\epsilon)^{\frac{1}{n-1}}$. Compute Δ_A and Δ_B according to (1) and (2), respectively.
2 Partition $[0, \Delta_A]$ into $\eta_A + 1$ subintervals $I_0^A = [0, 1)$, $I_1^A = [1, \delta)$, $I_2^A = [\delta, \delta^2)$, $\cdots$, $I_{\eta_A-1}^A = [\delta^{\eta_A-2}, \delta^{\eta_A-1})$, $I_{\eta_A}^A = [\delta^{\eta_A-1}, \Delta_A]$, where $\eta_A = \lceil \log_\delta \Delta_A \rceil = O(\frac{n \log \Delta_A}{\epsilon})$.
3 Partition $[0, \Delta_B]$ into $\eta_B + 1$ subintervals $I_0^B = [0, 1)$, $I_1^B = [1, \delta)$, $I_2^B = [\delta, \delta^2)$, $\cdots$, $I_{\eta_B-1}^B = [\delta^{\eta_B-2}, \delta^{\eta_B-1})$, $I_{\eta_B}^B = [\delta^{\eta_B-1}, \Delta_B]$, where $\eta_B = \lceil \log_\delta \Delta_B \rceil = O(\frac{n \log \Delta_B}{\epsilon})$.
4 Renumber jobs in $\mathcal{J}_i^A$ in the SPT order and jobs in $\mathcal{J}_i^B$ in the EDD order, $i = 1, 2, \ldots, m$.
5 $\mathcal{L}_\epsilon(0, 0, 0) \leftarrow \{(0, 0)\}$, $\mathcal{L}_\epsilon(i, j_A, j_B) \leftarrow \emptyset$ for $i = 1, 2, \ldots, m$, $j_A = 0, 1, \ldots, n_i^A$, $j_B = 0, 1, \ldots, n_i^B$
6 **for** $i = 1$ **to** m **do**
7 $\mathcal{L}_\epsilon(i, 0, 0) \leftarrow \{(0, q_A, q_B) : (t, q_A, q_B) \in \mathcal{L}(i-1, n_{i-1}^A, n_{i-1}^B)\}$
8 **for** $j_A = 0$ **to** n_i^A **do**
9 **for** $j_B = 0$ **to** n_i^B **do**
10 Run lines 7-21 in Algorithm **DP-TC-TG** to generate $\mathcal{L}_\epsilon(i, j_A, j_B)$.
11 Among all elements (t, q_A, q_B) in $\mathcal{L}_\epsilon(i, j_A, j_B)$ whose (q_A, q_B) falls into the same box $I_r^A \times I_l^B$, retain only the element with minimal t value.
12 **end**
13 **end**
14 **end**
15 Extract all PO points from $\mathcal{L}_\epsilon(m, n_m^A, n_m^B)$ to form set $\mathcal{X}_\epsilon$, and generate a corresponding schedule $\sigma(q_A, q_B)$ via backtracking for each $(q_A, q_B) \in \mathcal{X}_\epsilon$.

Output: $\mathcal{X}_\epsilon$ and $\sigma(q_A, q_B)$ for each $(q_A, q_B) \in \mathcal{X}_\epsilon$

The key difference between Algorithm **DP-TC-TC(ϵ)** and Algorithm **DP-TC-TG(ϵ)** lies in the partitioning step. In Algorithm **DP-TC-TG(ϵ)**, the range of both agent's objective function values is divided into geometrically increasing intervals. This partitioning

strategy allows for more effective approximation of the PO frontier and is crucial for ensuring that the algorithm remains efficient while generating the desired approximations.

Lemma 5.2. *For every state $(t, q_A, q_B) \in \mathcal{L}(i, j_A, j_B)$ generated by Algorithm **DP-TC-TG**, there exists a state $(\widetilde{t}, \widetilde{q_A}, \widetilde{q_B}) \in \mathcal{L}_\epsilon(i, j_A, j_B)$ with $\widetilde{t} \leq t$, $\widetilde{q_A} \leq \delta^{j-1} q_A$, and $\widetilde{q_B} \leq \delta^{j-1} q_B$ in Algorithm **DP-TC-TG(ϵ)**, where $j = N_{i-1} + j_A + j_B$.*

Proof. We prove the statement by induction on j from 1 to n. For the base case $j = 1$, three scenarios exist in both Algorithms **DP-TC-TC** and **DP-TC-TG(ϵ)**: (i) $J^A_{1,1}$ is the first scheduled valid job on M_1, (ii) $J^B_{1,1}$ is the first scheduled valid job on M_1, (iii) $J^B_{1,1}$ is the first scheduled job on M_1 and it is invalid. From the execution of these two algorithms, we have $\mathcal{L}_\epsilon(1,1,0) = \mathcal{L}(1,1,0) = \{(0, p^A_{1,1}, 0)\}$, $\mathcal{L}_\epsilon(1,0,1) = \mathcal{L}(1,0,1) = \{(p^B_{1,1}, 0, \psi(1,1,B,p^B_{1,1})), (0,0,\omega(1,1,B))\}$ if $p^B_{1,1} \leq \varphi(1,1,B)$, and $\mathcal{L}_\epsilon(1,0,1) = \mathcal{L}(1,0,1) = \{(0,0,\omega(1,1,B))\}$ if $p^B_{1,1} > \varphi(1,1,B)$. Thus, the statement is true for $j = 1$.

Now consider the scenario where $j \geq 2$ jobs has been scheduled during the algorithm and assume that the statement holds for $j-1$. Note that for a given $j \geq 2$, there exists a unique machine index $i \in \{1, 2, \ldots, m\}$ such that $N_{i-1} < j \leq N_i$. Let $(t, q_A, q_B) \in \mathcal{L}(i, j_A, j_B)$ be a state generated by Algorithm **DP-TC-TG** for jobs in $\mathcal{N}_{i-1} \cup \mathcal{J}^A_{i,j_A} \cup \mathcal{J}^B_{i,j_B}$, where $j = N_{i-1} + j_A + j_B$. The state $(t, q_A, q_B) \in \mathcal{L}(i, j_A, j_B)$ may be generated in Algorithm **DP-TC-TG** through three distinct cases.

Case 1. J^A_{i,j_A} is the last scheduled valid job on M_i (assuming $j_A \geq 1$). In this context, according to the execution of Algorithm **DP-TC-TG**, there exists a state $(t, q'_A, q_B) \in \mathcal{L}(i, j_A - 1, j_B)$, where $q'_A = q_A - t - P^A_{i,j_A}$. Based on the induction hypothesis, there exists a state $(\overline{t}, \overline{q'_A}, \overline{q_B}) \in \mathcal{L}_\epsilon(i, j_A - 1, j_B)$ with $\overline{t} \leq t$, $\overline{q'_A} \leq \delta^{j-2} q'_A$ and $\overline{q_B} \leq \delta^{j-2} q_B$ in Algorithm **DP-TC-TG(ϵ)**. Since $j_A \geq 1$, the state $(\overline{t}, \overline{q'_A} + \overline{t} + P^A_{i,j_A}, \overline{q_B})$ is added to $\mathcal{L}_\epsilon(i, j_A, j_B)$. However, this state may be eliminated in Algorithm **DP-TC-TG(ϵ)** (line 11) by another state $(\widetilde{t}, \widetilde{q_A}, \widetilde{q_B}) \in \mathcal{L}_\epsilon(i, j_A, j_B)$. For the state $(\widetilde{t}, \widetilde{q_A}, \widetilde{q_B})$, according to the execution of Algorithm **DP-TC-TG(ϵ)**, we get $\widetilde{t} \leq \overline{t} \leq t$, $\widetilde{q_A} \leq \delta(\overline{q'_A} + \overline{t} + P^A_{i,j_A}) \leq \delta(\delta^{j-2} q'_A + t + P^A_{i,j_A}) < \delta^{j-1}(q'_A + t + P^A_{i,j_A}) = \delta^{j-1} q_A$, and $\widetilde{q_B} \leq \delta \overline{q_B} \leq \delta^{j-1} q_B$. This establishes that the statement is true when J^A_{i,j_A} is the last scheduled valid job on M_i.

Case 2. J^B_{i,j_B} is the last scheduled valid job on M_i (assuming $j_B \geq 1$ and $t + P^A_{i,j_A} \leq \varphi(i, j_B, B)$). In this context, according to the execution of Algorithm **DP-TC-TG**, there exists a state $(t', q_A, q'_B) \in \mathcal{L}(i, j_A, j_B - 1)$, where $t' = t - p^B_{i,j_B}$ and $q'_B = q_B - \psi(i, j_B, B, t + P^A_{i,j_A})$. Based on the induction hypothesis, there exists a state $(\overline{t'}, \overline{q_A}, \overline{q'_B}) \in \mathcal{L}_\epsilon(i, j_A, j_B - 1)$ with $\overline{t'} \leq t'$, $\overline{q_A} \leq \delta^{j-2} q_A$ and $\overline{q'_B} \leq \delta^{j-2} q'_B$ in Algorithm **DP-TC-TG(ϵ)**. Since $j_B \geq 1$ and $\overline{t'} + P^A_{i,j_A} + p^B_{i,j_B} \leq t + P^A_{i,j_A} \leq \varphi(i, j_B, B)$, the state $(\overline{t'} + p^B_{i,j_B}, \overline{q_A}, \overline{q'_B} + \psi(i, j_B, B, \overline{t'} + P^A_{i,j_A} + p^B_{i,j_B}))$ is added to $\mathcal{L}_\epsilon(i, j_A, j_B)$. However, this state may be eliminated in Algorithm **DP-TC-TG(ϵ)** (line 11) by another state $(\widetilde{t}, \widetilde{q_A}, \widetilde{q_B}) \in \mathcal{L}_\epsilon(i, j_A, j_B)$. For the state $(\widetilde{t}, \widetilde{q_A}, \widetilde{q_B})$, according to the execution of Algorithm **DP-TC-TG(ϵ)** and the fact that function $\psi(i, j_B, B, \cdot)$ is nondecreasing, we get $\widetilde{t} \leq \overline{t'} + p^B_{i,j_B} \leq t' + p^B_{i,j_B} = t$, $\widetilde{q_A} \leq \delta \overline{q_A} \leq \delta^{j-1} q_A$, and $\widetilde{q_B} \leq$

$\delta(\overline{q'_B} + \psi(i, j_B, B, \overline{t'} + P^A_{i,j_A} + p^B_{i,j_B})) \leq \delta(\delta^{j-2}q'_B + \psi(i, j_B, B, t' + P^A_{i,j_A} + p^B_{i,j_B})) \leq \delta^{j-1}(q'_B + \psi(i, j_B, B, t + P^A_{i,j_A})) = \delta^{j-1}q_B$. This establishes that the statement is true when J^B_{i,j_B} is the last scheduled valid job on M_i.

Case 3. J^B_{i,j_B} is the last scheduled job on M_i and it is invalid (assuming $j_B \geq 1$). In this context, according to the execution of Algorithm **DP-TC-TG**, there exists a state $(t, q_A, q'_B) \in \mathcal{L}(i, j_A, j_B - 1)$, where $q'_B = q_B - \omega(i, j_B, B)$. Based on the induction hypothesis, there exists a state $(\bar{t}, \overline{q_A}, \overline{q'_B}) \in \mathcal{L}_\epsilon(i, j_A, j_B - 1)$ with $\bar{t} \leq t$, $\overline{q_A} \leq \delta^{j-2}q_A$ and $\overline{q'_B} \leq \delta^{j-2}q'_B$ in Algorithm **DP-TC-TG(ϵ)**. Since $j_B \geq 1$, the state $(\bar{t}, \overline{q_A}, \overline{q'_B} + \omega(i, j_B, B))$ is added to $\mathcal{L}_\epsilon(i, j_A, j_B)$. However, this state may be eliminated in Algorithm **DP-TC-TG(ϵ)** (line 11) by another state $(\widetilde{t}, \widetilde{q_A}, \widetilde{q_B}) \in \mathcal{L}_\epsilon(i, j_A, j_B)$. For the state $(\widetilde{t}, \widetilde{q_A}, \widetilde{q_B})$, according to the execution of Algorithm **DP-TC-TG(ϵ)**, we get $\widetilde{t} \leq \bar{t} \leq t$, $\widetilde{q_A} \leq \delta\overline{q_A} \leq \delta^{j-1}q_A$, and $\widetilde{q_B} \leq \delta(\overline{q'_B} + \omega(i, j_B, B)) \leq \delta(\delta^{j-2}q'_B + \omega(i, j_B, B)) \leq \delta^{j-1}(q'_B + \omega(i, j_B, B)) = \delta^{j-1}q_B$. This establishes that the statement is true when J^B_{i,j_B} is the last scheduled job on M_i and it is invalid.

Thus, the induction hypothesis is true in all three case. □

Theorem 5.2. *Algorithm **DP-TC-TG(ϵ)** is an FPTAS for problem $PDm|CO|\#(\mathrm{TC}^A, \mathrm{TG}^B)$, which constructs a $(1+\epsilon, 1+\epsilon)$-approximate PO frontier in $O(\frac{n^2 n_A n_B \log \Delta_A \log \Delta_B}{\epsilon^2})$ time.*

Proof. For every PO point $(q_A, q_B) \in \mathcal{X} \subseteq \mathcal{L}(m, n^A_m, n^B_m)$, utilizing the result of Lemma 5.2 and noting that $n = N_{m-1} + n^A_m + n^B_m$, Algorithm **DP-TC-TG($\epsilon$)** generates a state $(\widetilde{q_A}, \widetilde{q_B}) \in \mathcal{L}_\epsilon(m, n^A_m, n^B_m)$ such that $\widetilde{q_A} \leq q_A$ and $\widetilde{q_B} \leq \delta^{n-1}q_B$. Since $\delta = (1+\epsilon)^{\frac{1}{n-1}}$, we have $\widetilde{q_B} \leq (1+\epsilon)q_B$. Base on the execution of line 15 in Algorithm **DP-TC-TG(ϵ)**, we can find a state $(\widehat{q_A}, \widehat{q_B}) \in \mathcal{X}_\epsilon$ with $\widehat{q_A} \leq \widetilde{q_A}$ and $\widehat{q_B} \leq (1+\epsilon)\widetilde{q_A}$. Consequently, Algorithm **DP-TC-TG(ϵ)** constructs a $(1, 1+\epsilon)$-approximate PO frontier for problem $PDm|CO|\#(\mathrm{TC}^A, \mathrm{TG}^B)$.

Next, the running time of Algorithm **DP-TC-TG(ϵ)** is estimated. The interval partition takes $O(\eta_A + \eta_B) = O(\frac{n(\log \Delta_A + \log \Delta_B)}{\epsilon})$ time (lines 1-3). The renumbering operation requires $O(n_A \log n_A + n_B \log n_B)$ time (line 4). The initialization takes $O(n_A n_B)$ time (line 5). During the generation process (lines 6-14), at most $O(\eta_A \eta_B) = O(\frac{n^2 \log \Delta_A \log \Delta_B}{\epsilon^2})$ distinct states are retained in $\mathcal{L}_\epsilon(i, j_A, j_B)$ due to the elimination operation (line 11), and each state generates up to three new states. Hence, generating each $\mathcal{L}_\epsilon(i, j_A, j_B)$ can be accomplished in $O(\frac{n^2 \log \Delta_A \log \Delta_B}{\epsilon^2})$ time. Consequently, the entire generation process takes $\sum_{i=1}^m O(\frac{n^2 n_i^A n_i^B \log \Delta_A \log \Delta_B}{\epsilon^2}) = O(\frac{n^2 n_A n_B \log \Delta_A \log \Delta_B}{\epsilon^2})$ time, which dominates the time required for extraction and backtracking (line 15). Therefore, the total running time is $O(\frac{n^2 n_A n_B \log \Delta_A \log \Delta_B}{\epsilon^2})$. □

In the case where $\mathrm{TG}^B = TU^B$, we can derive a simplified result by ignoring line 3 and modifying the elimination operation in line 11 of Algorithm **DP-TC-TG(ϵ)**.

Remark 5.2. *For problem $PDm|CO|\#(\mathrm{TC}^A, \mathrm{TU}^B)$, there exists an FPTAS that constructs a $(1+\epsilon, 1)$-approximate PO frontier in $O(\frac{n n_A n_B^2 \log \Delta_A}{\epsilon})$ time.*

5.3 Problem $PDm|CO|\#(\mathrm{TG}^A, \mathrm{TG}^B)$

Building upon the exact DP algorithm in Section 4.3 and the algorithmic frameworks established in Sections 5.1 and 5.2, we develop an FPTAS for problem $PDm|CO|\#(\mathrm{TG}^A, \mathrm{TG}^B)$.

Algorithm DP-TG-TG(ϵ): An FPTAS for $PDm|CO|\#(TG^A, TG^B)$

Input: m, n, ϵ, $\mathcal{J}_i^X$, n_i^X, $p_{i,j}^X$, $d_{i,j}^X$, $w_{i,j}^X$ for $X = A, B$, $i = 1, 2, \ldots, m$ and $j = 1, 2, \ldots, n_i^X$

1 Run lines 1-3 in Algorithm **DP-TC-TG(ϵ)**.
2 Renumber jobs in $\mathcal{J}_i^A$ and $\mathcal{J}_i^B$ in their respective SPT order for $i = 1, 2, \ldots, m$.
3 $\mathcal{L}_\epsilon(0, 0, 0) \leftarrow \{(0, 0)\}$, $\mathcal{L}_\epsilon(i, j_A, j_B) \leftarrow \emptyset$ for $i = 1, 2, \ldots, m$, $j_A = 0, 1, \ldots, n_i^A$, $j_B = 0, 1, \ldots, n_i^B$
4 **for** $i = 1$ **to** m **do**
5 $\mathcal{L}_\epsilon(i, 0, 0) \leftarrow \{(0, q_A, q_B) : (t, q_A, q_B) \in \mathcal{L}(i-1, n_{i-1}^A, n_{i-1}^B)\}$
6 **for** $j_A = 0$ **to** n_i^A **do**
7 **for** $j_B = 0$ **to** n_i^B **do**
8 Run lines 7-26 in Algorithm **DP-TG-TG** to generate $\mathcal{L}_\epsilon(i, j_A, j_B)$.
9 Among all elements (t, q_A, q_B) in $\mathcal{L}_\epsilon(i, j_A, j_B)$ whose (q_A, q_B) falls into the same box $I_r^A \times I_l^B$, retain only the element with minimal t value.
10 **end**
11 **end**
12 **end**
13 Extract all PO points from $\mathcal{L}_\epsilon(m, n_m^A, n_m^B)$ to form set $\mathcal{X}_\epsilon$, and generate a corresponding schedule $\sigma(q_A, q_B)$ via backtracking for each $(q_A, q_B) \in \mathcal{X}_\epsilon$.

Output: $\mathcal{X}_\epsilon$ and $\sigma(q_A, q_B)$ for each $(q_A, q_B) \in \mathcal{X}_\epsilon$

Lemma 5.3. *For every state $(t, q_A, q_B) \in \mathcal{L}(i, j_A, j_B)$ generated by Algorithm **DP-TG-TG**, there exists a state $(\widetilde{t}, \widetilde{q_A}, \widetilde{q_B}) \in \mathcal{L}_\epsilon(i, j_A, j_B)$ with $\widetilde{t} \le t$, $\widetilde{q_A} \le \delta^{j-1} q_A$, and $\widetilde{q_B} \le \delta^{j-1} q_B$ in Algorithm **DP-TG-TG(ϵ)**, where $j = N_{i-1} + j_A + j_B$.*

Proof. The proof follows an analogous framework to that of Lemma 5.2. □

Theorem 5.3. *Algorithm **DP-TG-TG(ϵ)** is an FPTAS for problem $PDm|CO|\#(\mathrm{TG}^A, \mathrm{TG}^B)$, which constructs a $(1+\epsilon, 1+\epsilon)$-approximate PO frontier in $O(\frac{n^2 n_A n_B \log \Delta_A \log \Delta_B}{\epsilon^2})$ time.*

Proof. The proof follows an analogous framework to that of Theorem 5.2. □

Similar to the analysis of Remark 5.2, the following result holds.

Remark 5.3. *For problem $PDm|CO|\#(\mathrm{TY}^A, \mathrm{TU}^B)$, there exists an FPTAS that constructs a $(1+\epsilon, 1)$-approximate PO frontier in $O(\frac{n n_A n_B^2 \log P(A)}{\epsilon})$ time.*

5.4 Problem $PDm|CO|\#(\mathrm{TWU}^A, \mathrm{TWU}^B)$

We convert the exact DP algorithm designed in Section 4.4 for problem $PDm|CO|\#(\mathrm{TWU}^A, \mathrm{TWU}^B)$ into an FPTAS, which constructs a $(1+\epsilon, 1+\epsilon)$-approximate PO frontier.

Algorithm DP-I-TWU(ϵ): An FPTAS for $PDm|CO|\#(TWU^A, TWU^B)$

Input: m, n, ϵ, $\mathcal{J}_i$, n_i, $p_{i,j}^X$, $d_{i,j}^X$, $w_{i,j}^X$ for $X = A, B$, $i = 1, 2, \ldots, m$ and $j = 1, 2, \ldots, n_i^X$

Run lines 1-3 in Algorithm **DP-TC-TG(ϵ)**.
Renumber jobs in $\mathcal{J}_i$ in the EDD order such that $d_{i,1} \leq d_{i,2} \leq \cdots \leq d_{i,n_i}$, $i = 1, 2, \ldots, m$.
$\mathcal{L}_\epsilon(0,0) \leftarrow \{(0,0,0)\}$, $\mathcal{L}_\epsilon(i,j) \leftarrow \emptyset$ for $i = 1, 2, \ldots, m$, $j = 0, 1, \ldots, n_i$
for $i = 1$ **to** m **do**
    $\mathcal{L}_\epsilon(i,0) \leftarrow \{(0, q_A, q_B) : (t, q_A, q_B) \in \mathcal{L}(i-1, n_{i-1})\}$
    for $j = 1$ **to** n_i **do**
        Run lines 6-23 in Algorithm **DP-I-TWU** to generate $\mathcal{L}_\epsilon(i,j)$.
        Among all elements (t, q_A, q_B) in $\mathcal{L}_\epsilon(i,j)$ whose (q_A, q_B) falls into the same box $I_r^A \times I_l^B$, retain only the element with minimal t value.
    end
end
Extract all PO points from $\mathcal{L}_\epsilon(m, n_m)$ to form set $\mathcal{X}_\epsilon$, and generate a corresponding schedule $\sigma(q_A, q_B)$ via backtracking for each $(q_A, q_B) \in \mathcal{X}_\epsilon$.

Output: $\mathcal{X}_\epsilon$ and $\sigma(q_A, q_B)$ for each $(q_A, q_B) \in \mathcal{X}_\epsilon$

Lemma 5.4. *For every state $(t, q_A, q_B) \in \mathcal{L}(i,j)$ generated by Algorithm **DP-I-TWU**, there exists a state $(\widetilde{t}, \widetilde{q_A}, \widetilde{q_B}) \in \mathcal{L}_\epsilon(i,j)$ with $\widetilde{t} \leq t$, $\widetilde{q_A} \leq \delta^{k-1} q_A$, and $\widetilde{q_B} \leq \delta^{k-1} q_B$ in Algorithm **DP-I-TWU(ϵ)**, where $k = N_{i-1} + j$.*

Proof. The proof follows an analogous framework to that of Lemma 5.2. □

Theorem 5.4. *Algorithm **DP-I-TWU(ϵ)** is an FPTAS for problem $PDm|CO|\#(\mathrm{TWU}^A, \mathrm{TWU}^B)$, which constructs a $(1+\epsilon, 1+\epsilon)$-approximate PO frontier in $O(\frac{n^3 \log W(A) \log W(B)}{\epsilon^2})$ time.*

Proof. The proof follows an analogous framework to that of Theorem 5.2. □

Similar to the analysis of Remark 5.2, the following result holds.

Remark 5.4. *For problem $PDm|CO|\#(\mathrm{TWU}^A, \mathrm{TU}^B)$, there exists an FPTAS that constructs a $(1+\epsilon, 1)$-approximate PO frontier in $O(\frac{n^2 n_B \log W(A)}{\epsilon})$ time.*

Table 1: Results for two-agent Pareto scheduling on parallel dedicated machines

Problem	Optimal algorithm	Approximation scheme
$PDm\|CO\|\#(\mathrm{TC}^A, \mathrm{TC}^B)$	$O(n_A n_B \min\{\Delta_A, \Delta_B\})$	$O((nn_A n_B \min\{\log \Delta_A, \log \Delta_B\}/\epsilon)$
Reference	Theorem 4.1	Theorem 5.1 & Remark 5.1
$PDm\|CO\|\#(\mathrm{TC}^A, \mathrm{TY}^B)$	$O(n_A n_B P^B_{\max} P(B))$	$O((n^2 n_A n_B \log \Delta_A \log P(B))/\epsilon^2)$
Reference	Corollary 4.1	Theorem 5.2
$PDm\|CO\|\#(\mathrm{TC}^A, \mathrm{TWU}^B)$	$O(n_A n_B P^B_{\max} W(B))$	$O((n^2 n_A n_B \log \Delta_A \log W(B))/\epsilon^2)$
Reference	Corollary 4.2	Theorem 5.2
$PDm\|CO\|\#(\mathrm{TC}^A, \mathrm{TU}^B)$	$O(n_A n_B^2 P^B_{\max})$	$O((nn_A n_B^2 \log \Delta_A)/\epsilon)$
Reference	Corollary 4.3	Remark 5.2
$PDm\|CO\|\#(\mathrm{TY}^A, \mathrm{TY}^B)$	$O(n_A n_B P(A) P(B))$	$O((n^2 n_A n_B \log P(A) \log P(B))/\epsilon^2)$
Reference	Corollary 4.5	Theorem 5.3
$PDm\|CO\|\#(\mathrm{TY}^A, \mathrm{TWU}^B)$	$O(n_A n_B P(A) W(B))$	$O((n^2 n_A n_B \log P(A) \log W(B))/\epsilon^2)$
Reference	Corollary 4.6	Theorem 5.3
$PDm\|CO\|\#(\mathrm{TY}^A, \mathrm{TU}^B)$	$O(n_A n_B^2 P(A))$	$O((nn_A n_B^2 \log P(A))/\epsilon)$
Reference	Corollary 4.7	Remark 5.3
$PDm\|CO\|\#(\mathrm{TWU}^A, \mathrm{TWU}^B)$	$O(nW(A)W(B))$	$O((n^3 \log W(A) \log W(B))/\epsilon^2)$
Reference	Theorem 4.4	Theorem 5.4
$PDm\|CO\|\#(\mathrm{TWU}^A, \mathrm{TU}^B)$	$O(nn_B W(A))$	$O((n^2 n_B \log W(A))/\epsilon)$
Reference	Corollary 4.8	Remark 5.4
$PDm\|CO\|\#(\mathrm{TU}^A, \mathrm{TU}^B)$	$O(nn_A n_B)$	
Reference	Corollary 4.9	

References

Agnetis, A., Billaut, J. C., Gawiejnowicz, S., Pacciarelli, D., and Soukhal, A. (2014). *Multiagent Scheduling: Models and Algorithms*. Springer, Berlin, Heidelberg.

Agnetis, A., Mirchandani, P. B., Pacciarelli, D., and Pacifici, A. (2004). Scheduling problems with two competing agents. *Operations Research*, 52(2):229–242.

Boeckmann, J., Thielen, C., and Pferschy, U. (2023). Approximating single- and multi-objective nonlinear sum and product knapsack problems. *Discrete Optimization*, 48:100771.

Ibarra, O. H. and Kim, C. E. (1975). Fast approximation algorithms for the knapsack and sum of subset problems. *Journal of the ACM*, 22(4):463–468.

Karp, R. M. (1972). Reducibility among combinatorial problems. In Miller, R. E., Thatcher, J. W., and Bohlinger, J. D., editors, *Complexity of Computer Computations*, The IBM Research Symposia Series, pages 85–103. Springer, Boston, MA.

Li, S. S. and Yuan, J. J. (2020). Single-machine scheduling with multi-agents to minimize total weighted late work. *Journal of Scheduling*, 23(4):497–512.

Potts, C. N. and Van Wassenhove, L. N. (1992). Single machine scheduling to minimize total late work. *Operations Research*, 40(3):586–595.

T'kindt, V. and Billaut, J. C. (2006). *Multicriteria scheduling: Theory, Models and Algorithms*. Springer Berlin, Heidelberg, second edition.